

A.B.V.M. Agrawal Jatiya Kosh's
Ghanshyamdas Jalan College of Science & Commerce
Malad (East), Mumbai
(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY

Dot Net Core and Progressive Web Application

B.Sc. Information Technology

Submitted by

Ayesha Sarguru
Seat Number : 502520019

Under the Guidance of

Prof. Raviprakash Jaiswal

Academic Year 2026–27
October 2026



Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

Certificate by the Institute

Department of Information Technology

I hereby certify that **Ayesha Sarguru** , Seat Number : 502520019, a student of Ghanshyamdas Jalan College of Science & Commerce, Malad (East), studying in **B.Sc. Information Technology**, has successfully completed the practical work in **Dot Net Core and Progressive Web Application** for the academic year 2026–27, under the guidance of Prof. Raviprakash Jaiswal. To the best of my knowledge, the work of the student is original and the information included in the practical is correct.

Internal Guide	**External Examiner**	**Head of the Department**

Date: _____ **College Seal:**

INDEX

Sr. No.	Practical Name / Title	Page No.	Signature
MODULE 1			
1	Practical 1.a: Arithmetic Operations using ASP.NET Core MVC	1	
2	Practical 1.b: Floyd's Triangle in C#	7	
3	Practical 1.c: Sort the Names of 10 Cities	8	
4	Practical 2.a: Function Overloading	10	
5	Practical 2.b: Inheritance (All Types)	11	
6	Practical 2.c: Constructor Overloading	13	
7	Practical 2.d: Interfaces	14	
8	Practical 2.e: Delegates and Events	15	
9	Practical 2.f: Exception Handling	17	
10	Practical 3.a: Arithmetic Methods Using C# Delegates	19	
11	Practical 3.b: Multicast Delegate	20	
12	Practical 3.c: BankAccount Class with Balance Validation	21	
13	Practical 4.a: ASP.NET Web Forms Using Simple Server Controls	22	
14	Practical 4.b: Registration Form Using Validation Controls	26	
15	Practical 4.c: AdRotator Control	30	
16	Practical 5.a: Different Types of Cookies	33	
17	Practical 5.b: Product Table and Insert Operation Using Entity Framework Core	37	
18	Practical 5.c: CRUD Operations on Employee Table Using EF Core	43	
19	Practical 5.d: Authentication and Authorization in ASP.NET Core MVC	50	

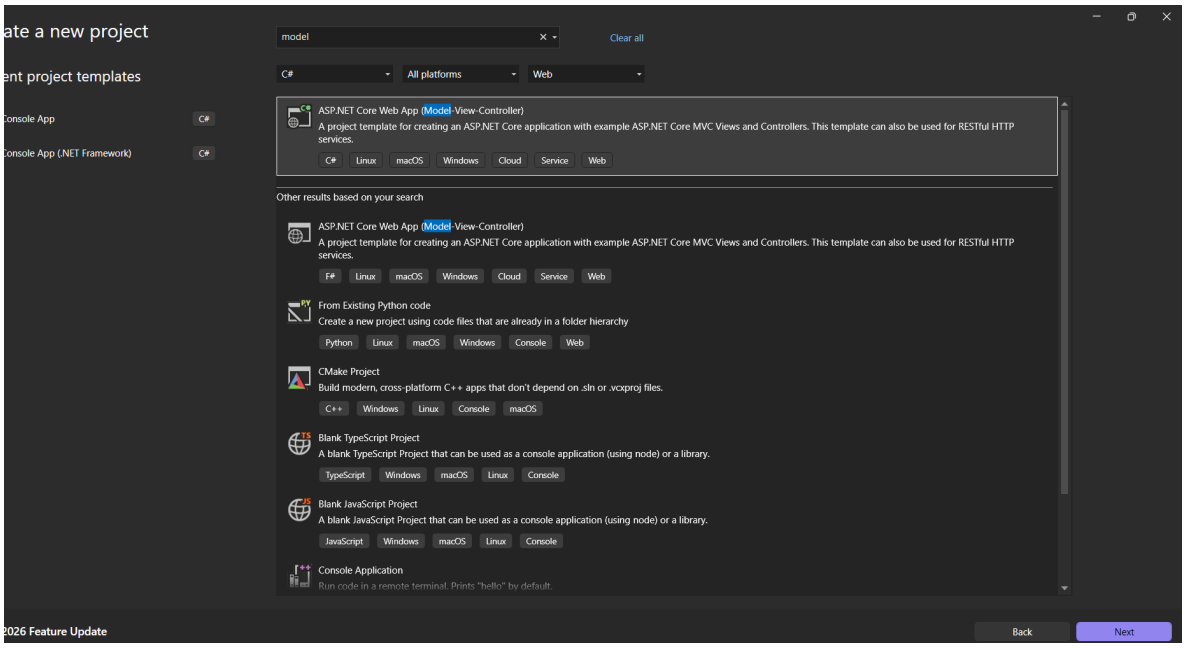
Sr. No.	Practical Name / Title	Page No.	Signature
MODULE 2			
20	Practical 6.a: College Information Portal Converted into a PWA	57	
21	Practical 6.b: PWA with a Custom Offline Page	61	
22	Practical 7.a: Offline PWA Using a Service Worker	68	
23	Practical 7.b: Student Record Management PWA	68	
24	Practical 8.a: Starting HTML Page Linked to manifest.json	89	
25	Practical 8.b: Student Web API Returning JSON Data	92	
26	Practical 9.a: CRUD Operations Using ASP.NET Core Web API and EF Core	95	
27	Practical 9.b: Factorial API Endpoint Tested Using Postman	103	
28	Practical 10.a: PWA Manifest with App Name, Icons, Start URL and Display Mode	106	
29	Practical 10.b: E-Commerce PWA with Product Listing and Offline Cart	108	
30	Practical 10.c: Implement Push Notifications in a PWA	118	

Practical 1.a

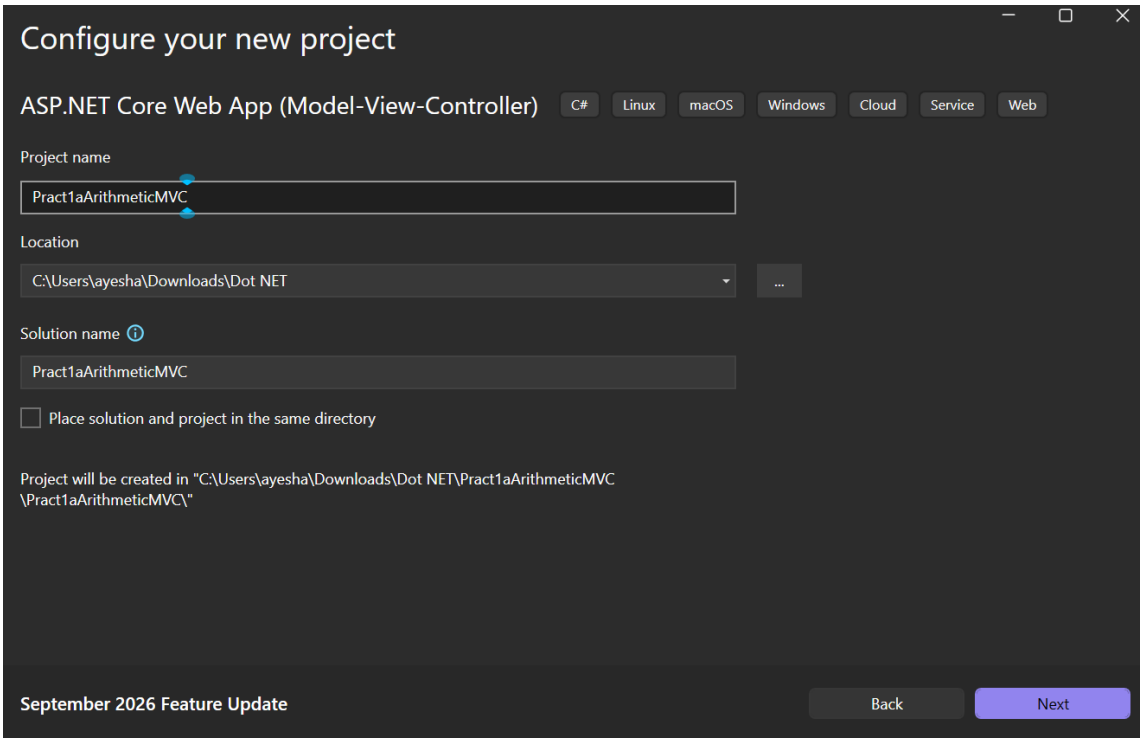
Aim: Create an MVC application to perform basic Arithmetic Operations.

Steps:

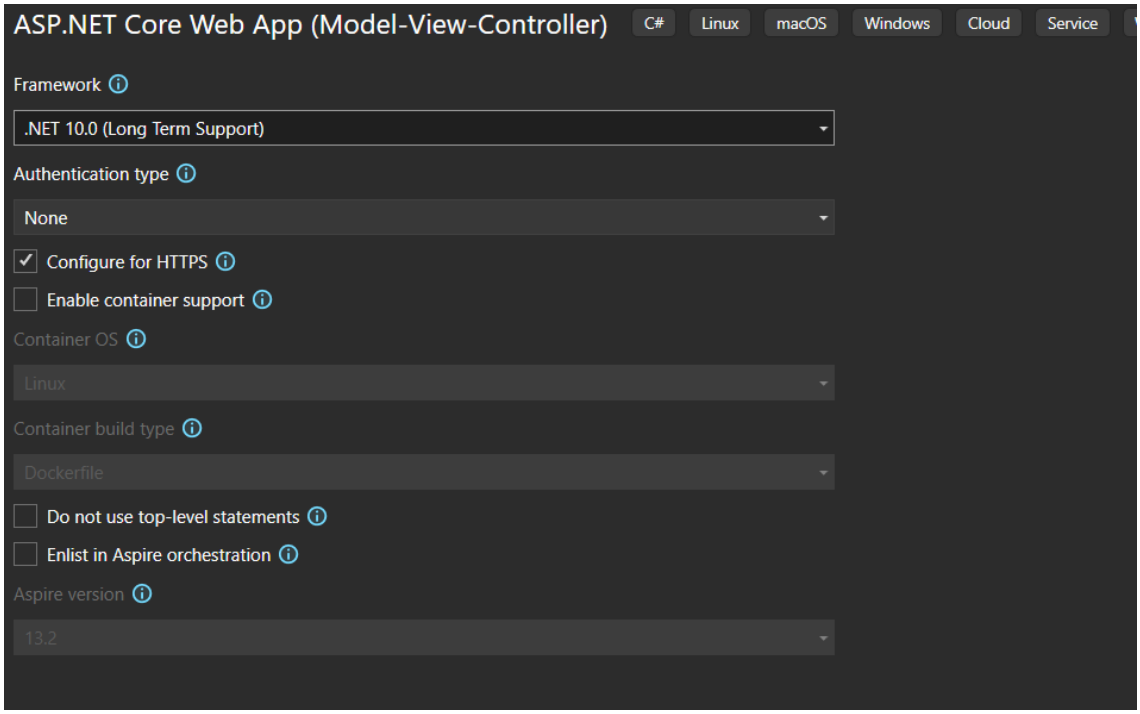
1. **Open** Visual Studio 2026
2. **Create** Create New Project →
3. **Search: Type Model View Controller**
4. **Select Type of Project → ASP.NET Core Web App(Model View Controller)**



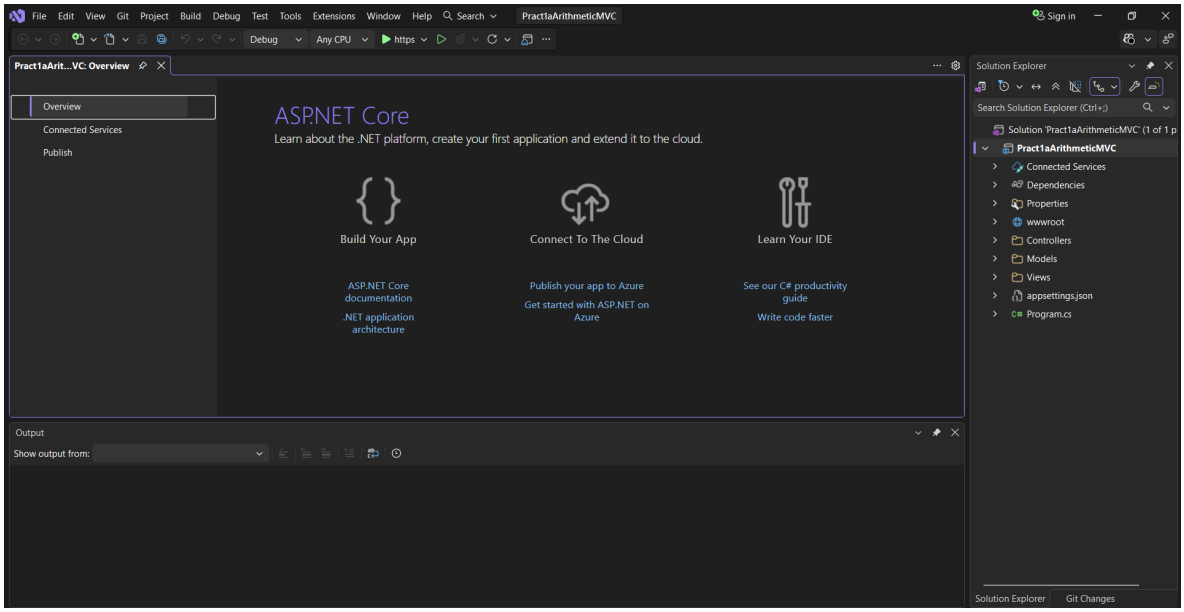
5. **Configure Project:**
   Project Name: Pract1aArithmeticMVC
   Loaction: C:\Ayesha.net
   Solution Name: Pract1aArithmeticMVC



6. **Additional Information:**
   Option



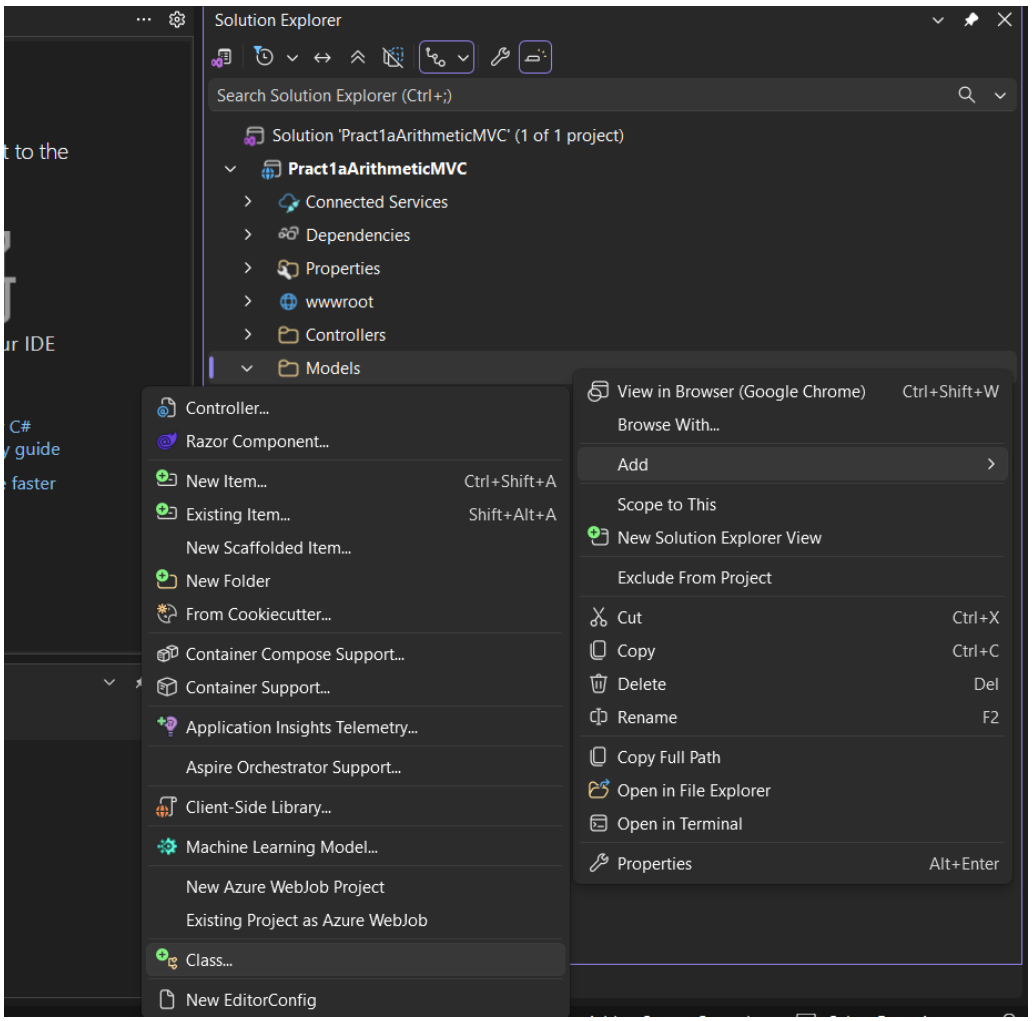
7. You can see the below screen:



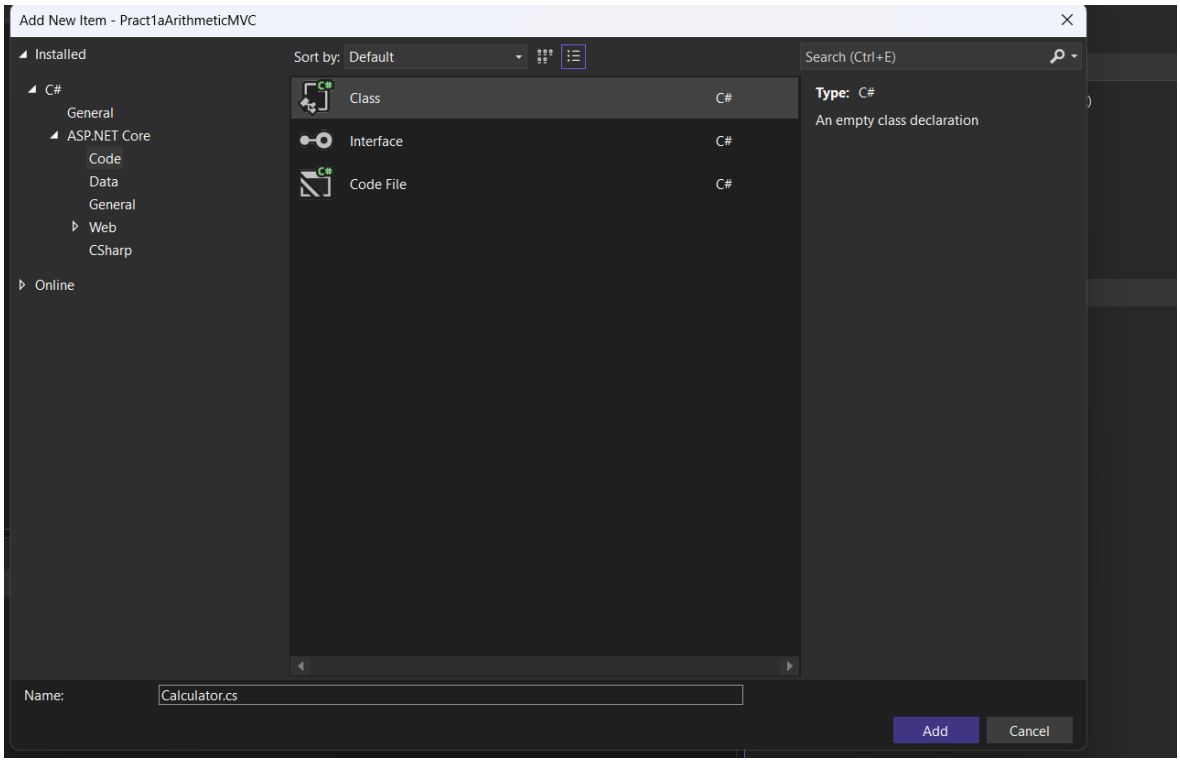
8. **Understand your Project:**
   On right side you will see in solution Explorer:
   Main Folder:
   Controller: Contains Controller Files
   Models: Contains Data Classes
   Views: Contains UI Pages
   WWWroot: CSS, JS, Images
   Program.cs: Main Start Up File

9. Create Model File:
   In Solution Explorer (If this window is not seen, go to View Menu and select it.)
   Right Click Models

Chose Add → Class



Enter the file Name → Calculator.cs → click Add



10. **Write Model Code (**Delete Existing Code and Type):
    Code:

```
namespace Pract1aArithmeticMVC.Models
{
    public class Calculator
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public required String Operation { get; set; }

        public double Result { get; set; }
    }
}
```

Press Ctrl+S to save

11. **Open Controller**

```
In Solution Explorer: Open: Controller => Double Click:
```

HomeController.cs

12. **Modify Controller Code:**

Remove existing Code and inside HomeController and type the following Code:

```
using Microsoft.AspNetCore.Mvc;
using Pract1aArithmeticMVC.Models;
using System.Diagnostics;

namespace Pract1aArithmeticMVC.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult Privacy()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Index(Calculator calc)
        {
            switch(calc.Operation)
            {
                case "+":
                    calc.Result = calc.Number1 + calc.Number2;
                        break;
                case "-":
                    calc.Result = calc.Number1 - calc.Number2;
                        break;
                case "*":
                    calc.Result = calc.Number1 * calc.Number2;
                        break;
                case "/":
                    if (calc.Number2 != 0)
                    {
                        calc.Result = calc.Number1 / calc.Number2;
                     }
                    break;
            }
            return View(calc);
        }
```

```
        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ??
HttpContext.TraceIdentifier });
        }


    }
}
```

Save the File.

13. Open View Folder:

```
In Solution Explorer: Open: Views => Home => Double Click:
```

Index.cshtml
Replace Existing View Code with the following Code:

```
@model Pract1aArithmeticMVC.Models.Calculator

<h2>Arithmetic Operations</h2>
<form action="/" method="post">
<label asp-for="Number1"> Enter Number 1:</label> <br/>
    <input type="number" asp-for="Number1" />
<br />
<br />
<label> Enter Number 2:</label>
<br />
    <input type="number" asp-for="Number2" />
<br />
<br />

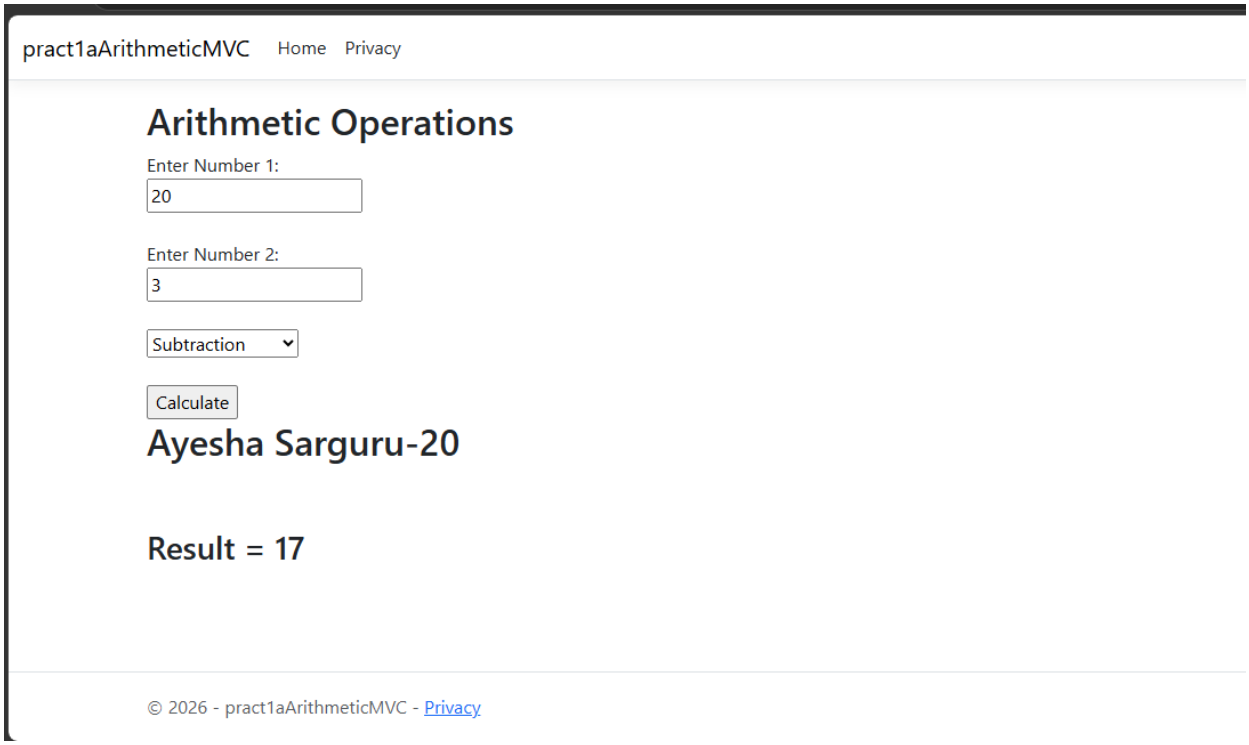

    <select asp-for="Operation">
    <option Value="op"> Select Operator </option>
    <option Value ="+"> Addition </option>
    <option Value="-"> Subtraction </option>
    <option Value ="*"> Multiplication </option>
    <option Value="/"> Division </option>
</select>
<br />
<br />
<input type="submit" value=" Calculate" />
</form>
<br />
<br />
@if(Model!=null)
{
<h3> Result = @Model.Result</h3>
}
```

Save the File.

14. **Build the Project:**
    At top menu click:
    Build → Build Solution
    Shortcut
    Ctrl + Shift +B
15. **Run the Application**
    Click:
    IIS Express
    Or Press Ctrl+F5

Browser will open automatically

Output:



Practical 1.b

Aim: Create an application to print Floyd's triangle till n rows in C#.

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Pract1bFloydTriangle name to your project.
3. Open Program.cs
4. Write the following code

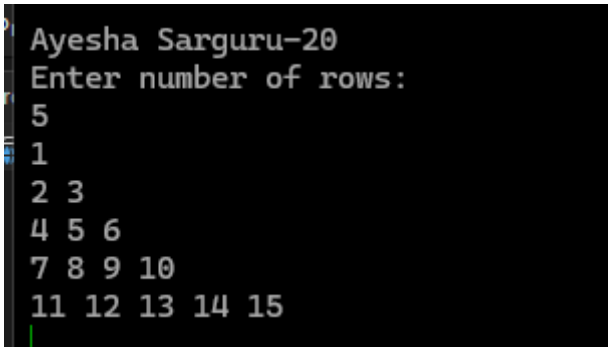
Code:

```
//Pract 1b: Create an application to print Floyd's triangle till n rows in C#.

using System;
//using System.Collections.Generic;
//using System.Linq;
//using System.Text;
//using System.Threading.Tasks;

namespace Pract1bFloydTriangle
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int n, number = 1;
            Console.Write("Enter number of rows:");
            n=Convert.ToInt32(Console.ReadLine());
            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(number + "    ");
                    number++;
                }
                Console.WriteLine();
                    }
            Console.ReadLine();
}}}
```

Output:



Practical 1.c

Aim: Write C# code to arrange the name of cities in sorted order. Accept name of 10 cities from the user

Steps:

1. Open Visual Studio 2026
2. Create → Console App, Give Pract1cArraySort name to your project.
3. Open Program.cs
4. Write the following code

Code:

```
/*Pract 1c:Write C# code to arrange the name of cities in sorted order. Accept name of 10
cities from the user . */

using System;
namespace Pract1cArraySort
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string[] cities = new string[10];
            Console.WriteLine("Enter 10 Cities");
            for (int i = 0; i < cities.Length; i++)
            {
                cities[i] = Console.ReadLine();
            }
              Array.Sort(cities);
               Console.WriteLine("\n Cities in sorted Order:");
               foreach (string city in cities)
                {
                        Console.WriteLine(city);
                }

        }
    }
}
```

Step 5: Build and Run your Program by pressing Ctrl +F5

Output:



Practical 2.a

Write the program with different features of C#

Aim: Create an application to Function Overloading Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Function Overloading name to your project.
3. Open Program.cs
4. Write the following code

   Code: -

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2aFuncOverloading
{
    class Addition
    {
        public void Add(int a, int b)
        {
            Console.WriteLine("Addition of two numbers = " + (a + b));
        }

        public void Add(double a, double b)
        {
            Console.WriteLine("Addition of two numbers = " + (a + b));
        }

        public void Add(int a, int b, int c)
        {
            Console.WriteLine("Addition of two numbers = " + (a + b + c));
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Addition obj = new Addition();

            obj.Add(10, 20);
            obj.Add(10.5, 20.5);
            obj.Add(10, 20, 30);

            Console.ReadLine();
        }
    }
}
```

Output:

```
Addition of two numbers = 30
Addition of two numbers = 31
Addition of two numbers = 60
```

Practical 2.b

Aim: Create an application to Inheritance (all types) Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Inheritance (all types) name to your project.
3. Open Program.cs
4. Write the following code

   Code: -

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2bInheritance
{
    //Single Inheritance
    class Animal
    {
        public void Eat()
        {
            Console.WriteLine("Animal can Eat");
        }
    }
    class Dog: Animal
    {
        public void Bark()
        {
            Console.WriteLine("Dog can bark");
        }
    }
    //Multilevel Inheritance
    class Puppy : Dog
    {
        public void Weep()
        {
            Console.WriteLine("\aPuppy can weep");
        }
    }
    //Hierarchical Inheritance
    class Cat : Animal
    {
        public void Meow()
        {
            Console.WriteLine("\aCat can meow .... meow");
        }
    }
    internal class Program
    {
        static void Main(string[] args)
        {
            //Single Inheritance
            Dog d = new Dog();
            d.Eat();
            d.Bark();

            //Multilevel Inheritance
            Puppy p = new Puppy();
            p.Eat();
            p.Bark();
            p.Weep();

            //Hierarchical Inheritance
            Cat c = new Cat();
            c.Eat();

            c.Meow();
        }
    }
}
```

Output:


Animal can Eat
Dog can bark
Animal can Eat
Dog can bark
Puppy can weep
Animal can Eat
Cat can meow .... meow
C:\Ayesha\Ayesha Dot Net\Pract2B\Pract2B\bin\Debug\net10.0\Pract2B.exe (process 21396) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
Open a new tab
Alt+Click to split the current window
Shift+Click to open a new window

Practical 2.c

Aim: Create an application to Constructor overloading Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Constructor overloading name to your project.
3. Open Program.cs
4. Write the following code

Code:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2cContructorOverlaoding
{
    class Students
    {
        //Defualt Constructor
        public Students()
        {
            Console.WriteLine("Default Constructor called");

        }
        //Parameterized Constructor
        public Students(string name)
        {
            Console.WriteLine("Student name:" + name);

        }

        //Another Parameterized Constructor
        public Students(string name, int age)
        {
            Console.WriteLine("Student name:" + name);
            Console.WriteLine("Student Age:" + age);

        }
    }
        internal class Program
    {
        static void Main(string[] args)
        {
            Students s1 = new Students();
            Students s2 = new Students("Ayesha");
            Students s3 = new Students("Ayesha",21);
            Console.ReadLine();


        }
    }
}
```

Output:



Practical 2.d

Aim: Create an application to Interfaces Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Interfaces name to your project.
3. Open Program.cs
4. Write the following code

Code:-

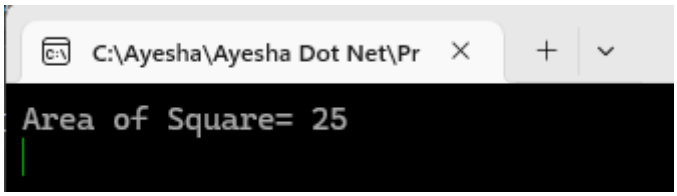
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2dInterface
{
    interface IShape
    {
        void Area();
    }

    class Sqaure : IShape
    {
        int side = 5;
        public void Area()
        {
            Console.WriteLine("Area of Square= " + (side * side));
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Sqaure sqaure = new Sqaure();
            sqaure.Area();
            Console.ReadLine();
        }
    }
}
```

Output:



Practical 2.e

Aim: Create an application to Using Delegates and events Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Pract2eDelegates to your project.
3. Open Program.cs
4. Write the following code

Code:-

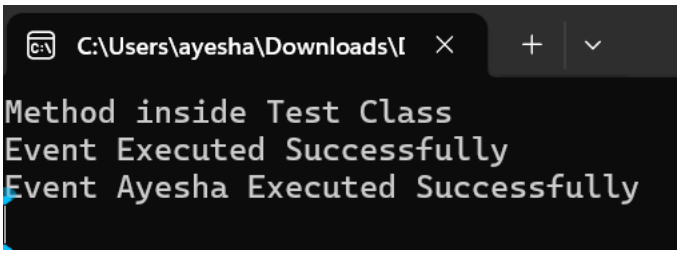
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2eDelegates
{
    public delegate void Message();
    class Test
    {
        public event Message Notify;

        public int Show()
        {
            Console.WriteLine("Method inside Test Class");
            //Raising Event
            Notify();
            return 0;
        }
    }
    internal class Program
    {
        //event Handling Method
        public  static void Display()
        {
            Console.WriteLine("Event Executed Successfully");
        }

        public static void Ayesha()
        {
            Console.WriteLine("Event Ayesha Executed Successfully");
        }
        public static int Ayeshaint()
        {
            Console.WriteLine("Event Ayesha Executed Successfully");
            return 0;
        }
        static void Main(string[] args)
        {
            Test t = new Test();
            //Attaching method to event
            t.Notify += Display;
            t.Notify += Ayesha;
            //t.Notify += Ayeshaint;
            t.Show();
            Console.ReadLine();
        }
    }
}
```

Output:


C:\Users\ayesha\Downloads\I
Method inside Test Class
Event Executed Successfully
Event Ayesha Executed Successfully

Practical 2.f

Aim: Create an application to Exception handling Method

Steps:

1. Open Visual Studio 2026
2. Create → Console App(.NET FrameWork), Give Exception handling to your project.
3. Open Program.cs
4. Write the following code

Code:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2fExceptionHandling
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a, b,c, result;
            try
            {
                Console.WriteLine("Enter First Number");
                a = Convert.ToInt32(Console.ReadLine());
                Console.WriteLine("Enter Second Number");
                b = Convert.ToInt32(Console.ReadLine());
                result = a / b;
                Console.WriteLine("Result = " + result);
                Console.WriteLine($"Result = {result}");
                int[] numbers = { 10, 20, 30, 40, 50 }; // Length is 5

                Console.WriteLine("which element do you want to see");
                c = Convert.ToInt32(Console.ReadLine());

                Console.WriteLine(numbers[c]); // Throws IndexOutOfRangeException


            }
            catch (IndexOutOfRangeException ex)
            {
                Console.WriteLine($"IndexOutOfRangeException:{ex.Message} & {ex.StackTrace}");
            }

            catch (DivideByZeroException ex)
            {
                Console.WriteLine("Can not divide by zero"+ex.Message+" & " + ex.StackTrace);
            }
            catch (FormatException)
            {
                Console.WriteLine("Invalid Input ");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Some Exception:{ex.Message} & {ex.StackTrace}");
            }

            finally
            {
                Console.WriteLine("Programm Ended");
            }
        }
    }
}
```

Output:

```
Enter First Number
15
Enter Second Number
3
Result = 5
Result = 5
which element do you want to see
2
30
Programm Ended
```

Practical 3

Aim:

a. Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

b. Write a program using multicast delegate.

c. Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Solution:

a) Arithmetic methods using a delegate

```
using System;

class Calculator
{
    // Methods
    public static int Add(int a, int b)
    {
        return a + b;
    }

    public static int Multiply(int a, int b)
    {
        return a * b;
    }

    public static int Subtract(int a, int b)
    {
        return a - b;
    }

    public static double Divide(int a, int b)
    {
        if (b == 0)
            throw new DivideByZeroException("Cannot divide by zero.");

        return (double)a / b;
    }

    // Delegate declaration
    public delegate double Operation(int a, int b);

    static void Main()
    {
        int x = 20;
        int y = 5;

        // Calling methods using delegate
        Operation op;

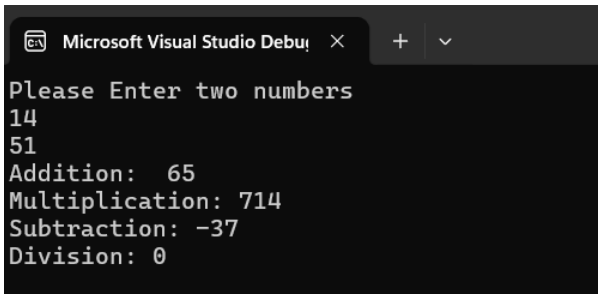
        op = Add;
        Console.WriteLine("Addition = " + op(x, y));

        op = Multiply;
        Console.WriteLine("Multiplication = " + op(x, y));

        op = Subtract;
        Console.WriteLine("Subtraction = " + op(x, y));

        op = Divide;
        Console.WriteLine("Division = " + op(x, y));
    }
}
```

Output:



b) Program using Multicast Delegate

A multicast delegate can call multiple methods using a single delegate object.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract3eDelegatesMulitcast
{
    using System;

    public class MulticastDelegateDemo
    {
        /*public delegate void Notify();

        public static void MethodA() => Console.WriteLine("Method A executed");
        public static void MethodB() => Console.WriteLine("Method B executed");

        public static void Main()
        {
            Notify notify = MethodA;
            notify += MethodB;

            notify(); // Invokes both MethodA and MethodB
            Console.WriteLine("After Removing Reference of MethodB");
            notify -= MethodB;

            notify(); // Invokes only MethodA

        }*/




        public delegate int Operation(int x, int y);

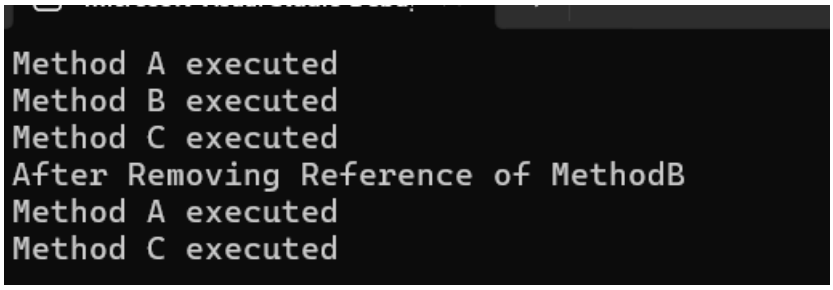
        public static int Add(int a, int b) => a+b;
        public static int Multiply(int a, int b) => a * b;
        public static int Subtract(int a, int b) => a - b;
        public static int Divide(int a, int b) => a / b;

        public static void Main()
        {
            Operation op = Add;
            Console.WriteLine($"Addition:  { op(5, 3)} ");

            op = Multiply;
            Console.WriteLine("Multiplication: " + op(5, 3));
            op = Subtract;
            Console.WriteLine("Subtraction: " + op(5, 3));
            op = Divide;
            Console.WriteLine("Division: " + op(5, 3));


        }
    }
}
```

Output:



c) BankAccount with Balance property validation

```
using System;

class BankAccount
{
    public string AccountNumber { get; set; }

    private double balance;

    public double Balance
    {
        get
        {
            return balance;
        }
        set
        {
            if (value < 0)
            {
                Console.WriteLine("Balance cannot be negative.");
            }
            else
            {
                balance = value;
            }
        }
    }

    static void Main()
    {
        BankAccount account = new BankAccount();

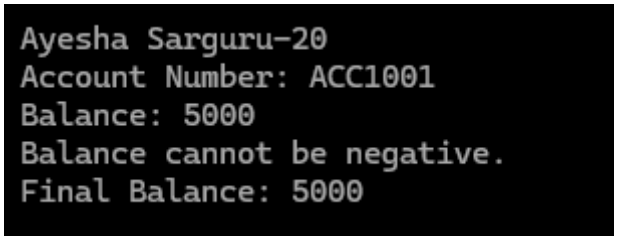
        account.AccountNumber = "ACC1001";

        account.Balance = 5000;
        Console.WriteLine("Account Number: " + account.AccountNumber);
        Console.WriteLine("Balance: " + account.Balance);

        // Trying to set negative balance
        account.Balance = -1000;

        Console.WriteLine("Final Balance: " + account.Balance);
    }
}
```

Output:



Practical 4.a

Student Registration Form

Aim: To create an ASP.NET Web Forms application using simple server controls such as Label, TextBox, RadioButton, DropDownList, CheckBox, Calendar, and Button.

Step 1: Create the project in Visual Studio 2026

1. Open **Visual Studio 2026**.
2. Select **Create a new project**.
3. Search for:
   ASP.NET Web Application (.NET Framework)
4. Select **ASP.NET Web Application (.NET Framework)**.
5. Click **Next**.
6. Enter:
   - **Project name:** StudentRegistration
   - Select your desired location.
7. Click **Create**.
8. Select **Web Forms**.
9. Click **Create**.

You will see files such as:

```
StudentRegistration
|
├── App_Start
├── Content
├── Scripts
├── Default.aspx
├── Default.aspx.cs
├── Site.Master
└── Web.config
```

Step 2: Design Student Registration Form

Open:

Default.aspx

Switch to **Source** view and replace the page contents with:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="Default.aspx.cs"
    Inherits="StudentRegistration.Default" %>

<!DOCTYPE html>

<html>
<head runat="server">
    <title>Student Registration</title>
</head>

<body>
<form id="form1" runat="server">

    <h2>Student Registration Form</h2>

    <asp:Label ID="lblName" runat="server"
        Text="Student Name:"></asp:Label>

    <asp:TextBox ID="txtName" runat="server"></asp:TextBox>

    <br /><br />

    <asp:Label ID="lblGender" runat="server"
```

```
        Text="Gender:"></asp:Label>

    <asp:RadioButton ID="rbFemale" runat="server"
        Text="Female" GroupName="Gender" />

    <asp:RadioButton ID="rbFeFemale" runat="server"
        Text="FeFemale" GroupName="Gender" />

    <br /><br />

    <asp:Label ID="lblCourse" runat="server"
        Text="Course:"></asp:Label>

    <asp:DropDownList ID="ddlCourse" runat="server">
        <asp:ListItem Text="-- Select Course --" Value=""></asp:ListItem>
        <asp:ListItem Text="BSc IT" Value="BSc IT"></asp:ListItem>
        <asp:ListItem Text="BSc CS" Value="BSc CS"></asp:ListItem>
        <asp:ListItem Text="BCA" Value="BCA"></asp:ListItem>
    </asp:DropDownList>

    <br /><br />

    <asp:Label ID="lblDate" runat="server"
        Text="Date of Birth:"></asp:Label>

    <br />

    <asp:Calendar ID="Calendar1" runat="server">
    </asp:Calendar>

    <br />

    <asp:CheckBox ID="chkTerms" runat="server"
        Text="I accept the Terms & Conditions" />

    <br /><br />

    <asp:Button ID="btnSubmit" runat="server"
        Text="Register"
        OnClick="btnSubmit Click" />

    <br /><br />

    <asp:Label ID="lblOutput" runat="server"
        ForeColor="Green">
    </asp:Label>

</form>
</body>
</html>
```

Step 3: Write C# code

Open:

Default.aspx.cs

Write:

```
using System;

namespace StudentRegistration
{
    public partial class Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnSubmit Click(object sender, EventArgs e)
        {
            string name = txtName.Text;

            string gender = "";

            if (rbFemale.Checked)
                gender = "Female";
            else if (rbFeFemale.Checked)
```

```
                gender = "FeFemale";
            else
                gender = "Not Selected";

            string course = ddlCourse.SelectedValue;

            string date = Calendar1.SelectedDate.ToShortDateString();

            if (!chkTerms.Checked)
            {
                lblOutput.ForeColor = System.Drawing.Color.Red;
                lblOutput.Text = "Please accept Terms & Conditions.";
                return;
            }

            lblOutput.ForeColor = System.Drawing.Color.Green;

            lblOutput.Text =
                "Registration Successful!<br/>" +
                "Student Name: " + name + "<br/>" +
                "Gender: " + gender + "<br/>" +
                "Course: " + course + "<br/>" +
                "Selected Date: " + date;
        }
    }
}
```

Step 4: Execute the practical

1. Save the project using:
   Ctrl + Shift + S

2. Build the project:
   Build → Build Solution

3. Press:
   Ctrl + F5
   or click the green **Start** button.

4. The application opens in the browser.
5. Enter:
6. Student Name: Ayesha
7. Gender: Female
8. Course: BSc IT
9. Select a date from the Calendar.
10. Check **I accept the Terms & Conditions**.
11. Click **Register**.

You should see something like:

Student Registration Form

Student Name Ayesha Sarguru

Gender ○ Male ◉ Female

Course Bsc IT

Date Of Birth

≤	February 2026					≥
Su	Mo	Tu	We	Th	Fr	Sa
25	26	27	28	29	30	31
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
1	2	3	4	5	6	7

☑ I accept the Terms & Conditions

Register

Registration Successful!
Student Name: Ayesha Sarguru
Gender: Female
Course: Bsc IT
Selected Date: 2/17/2026

Practical 4.b

Registration Form Using Validation Controls

Aim: To create an ASP.NET Web Form demonstrating various validation controls.

We will demonstrate:

- RequiredFieldValidator
- CompareValidator
- RangeValidator
- RegularExpressionValidator
- ValidationSummary
- Button

Step 1: Add a new Web Form

In **Solution Explorer**:

1. Right-click the project.
2. Select:
   Add → Web Form
3. Enter:

Validation.aspx

1. Click **Add**.

Step 2: Design Validation Form

Open:

Validation.aspx

Use **Source** view and write:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="Validation.aspx.cs"
    Inherits="StudentRegistration.Validation" %>

<!DOCTYPE html>

<html>
<head runat="server">
    <title>Registration Validation</title>
</head>

<body>

<form id="form1" runat="server">

    <h2>Registration Form</h2>

    <asp:ValidationSummary ID="ValidationSummary1"
        runat="server"
        ForeColor="Red" />

    <br />

    <asp:Label ID="Label1" runat="server"
        Text="Name:"></asp:Label>

    <asp:TextBox ID="txtName" runat="server"></asp:TextBox>

    <asp:RequiredFieldValidator
        ID="RequiredFieldValidator1"
```

```
        runat="server"
        ControlToValidate="txtName"
        ErrorMessage="Name is required."
        Text="*"
        ForeColor="Red">
    </asp:RequiredFieldValidator>

    <br /><br />

    <asp:Label ID="Label2" runat="server"
        Text="Age:"></asp:Label>

    <asp:TextBox ID="txtAge" runat="server"></asp:TextBox>

    <asp:RangeValidator
        ID="RangeValidator1"
        runat="server"
        ControlToValidate="txtAge"
        MinimumValue="18"
        MaximumValue="60"
        Type="Integer"
        ErrorMessage="Age must be between 18 and 60."
        Text="*"
        ForeColor="Red">
    </asp:RangeValidator>

    <br /><br />

    <asp:Label ID="Label3" runat="server"
        Text="Email:"></asp:Label>

    <asp:TextBox ID="txtEmail" runat="server"></asp:TextBox>

    <asp:RegularExpressionValidator
        ID="RegularExpressionValidator1"
        runat="server"
        ControlToValidate="txtEmail"
        ValidationExpression="\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*"
        ErrorMessage="Enter a valid email address."
        Text="*"
        ForeColor="Red">
    </asp:RegularExpressionValidator>

    <br /><br />

    <asp:Label ID="Label4" runat="server"
        Text="Password:"></asp:Label>

    <asp:TextBox ID="txtPassword" runat="server"
        TextMode="Password">
    </asp:TextBox>

    <asp:RequiredFieldValidator
        ID="RequiredFieldValidator2"
        runat="server"
        ControlToValidate="txtPassword"
        ErrorMessage="Password is required."
        Text="*"
        ForeColor="Red">
    </asp:RequiredFieldValidator>

    <br /><br />

    <asp:Label ID="Label5" runat="server"
        Text="Confirm Password:">
    </asp:Label>

    <asp:TextBox ID="txtConfirmPassword"
        runat="server"
        TextMode="Password">
    </asp:TextBox>

    <asp:CompareValidator
        ID="CompareValidator1"
        runat="server"
        ControlToValidate="txtConfirmPassword"
        ControlToCompare="txtPassword"
        ErrorMessage="Passwords do not match."
        Text="*"
```

```
        ForeColor="Red">
    </asp:CompareValidator>

    <br /><br />

    <asp:Button ID="btnRegister"
        runat="server"
        Text="Register"
        OnClick="btnRegister Click" />

    <br /><br />

    <asp:Label ID="lblMessage"
        runat="server"
        ForeColor="Green">
    </asp:Label>

</form>

</body>
</html>
```

Step 3: Code-behind

Open:

Validation.aspx.cs

Write:

```
using System;

namespace StudentRegistration
{
    public partial class Validation : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnRegister_Click(object sender, EventArgs e)
        {
            if (Page.IsValid)
            {
                lblMessage.Text =
                    "Registration Successful!";
            }
            else
            {
                lblMessage.Text =
                    "Please correct the errors.";
            }
        }
    }
}
```

Step 4: Execute

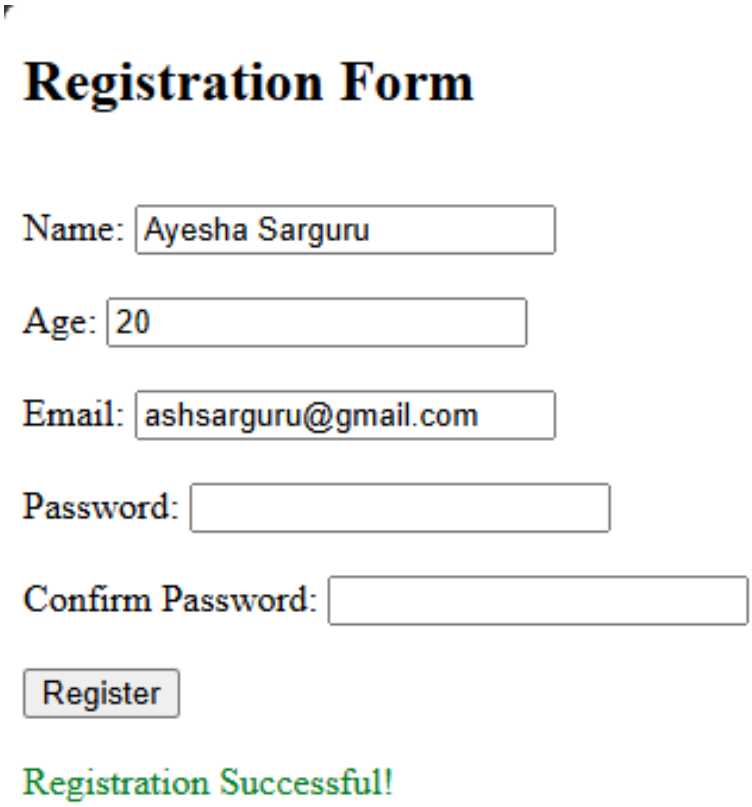
1. Right-click Validation.aspx.
2. Select:
   Set As Start Page

3. Press **Ctrl + F5**.
4. Test the form with invalid data.

For example:

```
Name:          [blank]
Age:           15
Email:         abc
Password:      12345
Confirm:       123
```

The validation controls will display errors.
Then enter valid information:



Practical 4.c

AdRotator Control

Aim: To create an ASP.NET Web Form demonstrating the use of the **AdRotator** control.

AdRotator displays advertisements based on an XML advertisement file.

Step 1: Add Web Form

In **Solution Explorer**:

Right-click Project → Add → Web Form

Name it:

AdRotator.aspx

Click **Add**.

Step 2: Create Advertisement XML file

Right-click the project:

Add → New Item

Select:

XML File

Name it:

Advertisements.xml

Click **Add**.

Put the following code in the XML file:

```
<?xml version="1.0" encoding="utf-8" ?>

<Advertisements>

  <Ad>
    <ImageUrl>~/Images/ad1.jpg</ImageUrl>
    <NavigateUrl>https://www.microsoft.com</NavigateUrl>
    <AlternateText>Microsoft Advertisement</AlternateText>
    <Impressions>50</Impressions>
    <Keyword>Technology</Keyword>
  </Ad>

  <Ad>
    <ImageUrl>~/Images/ad2.jpg</ImageUrl>
    <NavigateUrl>https://visualstudio.microsoft.com</NavigateUrl>
    <AlternateText>Visual Studio Advertisement</AlternateText>
    <Impressions>30</Impressions>
    <Keyword>Development</Keyword>
  </Ad>

  <Ad>
    <ImageUrl>~/Images/ad3.jpg</ImageUrl>
    <NavigateUrl>https://dotnet.microsoft.com</NavigateUrl>
    <AlternateText>.NET Advertisement</AlternateText>
    <Impressions>20</Impressions>
    <Keyword>Programming</Keyword>
  </Ad>

</Advertisements>
```

Step 3: Add Images folder

Right-click your project.

Select:

Add → New Folder

Name it:

```
Images
```

Add three images:

```
Images
├── ad1.jpg
├── ad2.jpg
└── ad3.jpg
```

Make sure the filenames exactly match the names in Advertisements.xml.
You can use your own advertisement images.

Step 4: Design AdRotator Web Form

Open:

AdRotator.aspx

Write:

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="AdRotator.aspx.cs"
    Inherits="StudentRegistration.AdRotator" %>

<!DOCTYPE html>

<html>
<head runat="server">
    <title>AdRotator Example</title>
</head>

<body>

<form id="form1" runat="server">

    <h2>Advertisement</h2>

    <asp:AdRotator
        ID="AdRotator1"
        runat="server"
        AdvertisementFile="~/Advertisements.xml"
        Width="500px"
        Height="200px" />

</form>

</body>
</html>
```

Step 5: Code-behind

Open:

AdRotator.aspx.cs

Use:

```
using System;

namespace StudentRegistration
{
    public partial class AdRotator : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }
    }
}
```

No additional C# code is required.

Step 6: Execute AdRotator

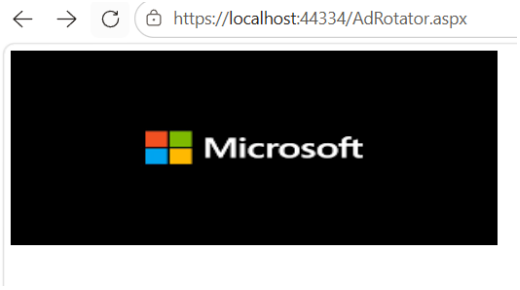
1. Right-click:
2. AdRotator.aspx
3. Select:
   Set As Start Page

4. Press:
   Ctrl + F5

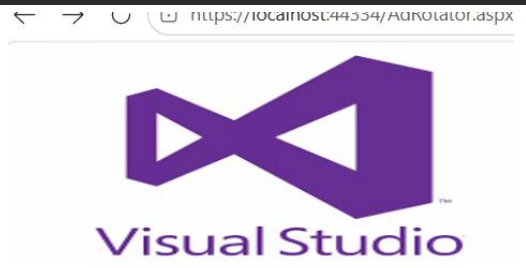
5. The advertisement will appear.
6. Refresh the page several times.

The AdRotator can display different advertisements based on their **Impressions** values.
For example:

```
ad1 → 50
```




```
ad2 → 30
```




```
ad3 → 20
```



Practical 5.a

Web Application to Demonstrate Different Types of Cookies

Aim: To create an ASP.NET Core MVC web application demonstrating:

1. Session Cookie
2. Persistent Cookie
3. Cookie with expiration
4. Reading a Cookie
5. Deleting a Cookie

Cookies are stored on the client and sent back with subsequent requests. (Microsoft Learn)

Step 1: Create ASP.NET Core MVC Project

Open **Visual Studio 2026**.

Select:

Create a new project

Search:

```
ASP.NET Core Web App (Model-View-Controller)
```

Select it and click **Next**.

Enter:

```
Project Name: CookieDemo
Location: Any suitable folder
```

Click **Next**.

For Framework, select:

```
.NET 10.0
```

Authentication:

```
None
```

Configure HTTPS:

```
Checked
```

Click **Create**.

Step 2: Project Structure

You will get:

```
CookieDemo
│
├── Controllers
│   └── HomeController.cs
│
├── Models
│
├── Views
│   ├── Home
│   │   └── Index.cshtml
│   └── Shared
│
├── wwwroot
│
├── Program.cs
└── appsettings.json
```

Step 3: Create Cookie Controller

Open:

```
Controllers → HomeController.cs
```

Replace the code with:

```
using Microsoft.AspNetCore.Mvc;

namespace CookieDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        // 1. Session Cookie
        public IActionResult CreateSessionCookie()
        {
            Response.Cookies.Append("UserName", "Ayesha");

            return Content("Session Cookie Created Successfully!");
        }

        // 2. Persistent Cookie
        public IActionResult CreatePersistentCookie()
        {
            CookieOptions options = new CookieOptions
            {
                Expires = DateTime.Now.AddDays(7)
            };

            Response.Cookies.Append(
                "PersistentUser",
                "Ayesha",
                options);

            return Content(
                "Persistent Cookie Created. It will expire after 7 days.");
        }

        // 3. Cookie with expiration
        public IActionResult CreateTemporaryCookie()
        {
            CookieOptions options = new CookieOptions
            {
                Expires = DateTime.Now.AddMinutes(5)
            };

            Response.Cookies.Append(
                "TemporaryCookie",
                "This cookie expires in 5 minutes",
                options);

            return Content("Temporary Cookie Created!");
        }

        // 4. Read Cookie
        public IActionResult ReadCookie()
        {
            string? userName =
                Request.Cookies["UserName"];

            if (userName == null)
            {
                return Content("Cookie does not exist.");
            }

            return Content(
                "Cookie Value: " + userName);
        }

        // 5. Delete Cookie
        public IActionResult DeleteCookie()
        {
```

```
            Response.Cookies.Delete("UserName");

            return Content("Cookie Deleted Successfully!");
        }
    }
}
```

Step 4: Design the Home Page

Open:

```
Views → Home → Index.cshtml
```

Replace it with:

```
@{
    ViewData["Title"] = "Cookie Demo";
}

<h1>Cookie Demonstration</h1>

<hr />

<h3>Create Cookies</h3>

<a asp-controller="Home"
   asp-action="CreateSessionCookie"
   class="btn btn-primary">
    Create Session Cookie
</a>

<br /><br />

<a asp-controller="Home"
   asp-action="CreatePersistentCookie"
   class="btn btn-success">
    Create Persistent Cookie
</a>

<br /><br />

<a asp-controller="Home"
   asp-action="CreateTemporaryCookie"
   class="btn btn-warning">
    Create Temporary Cookie
</a>

<hr />

<h3>Cookie Operations</h3>

<a asp-controller="Home"
   asp-action="ReadCookie"
   class="btn btn-info">
    Read Cookie
</a>

<br /><br />

<a asp-controller="Home"
   asp-action="DeleteCookie"
   class="btn btn-danger">
    Delete Cookie
</a>
```

Step 5: Run the Application

Press:

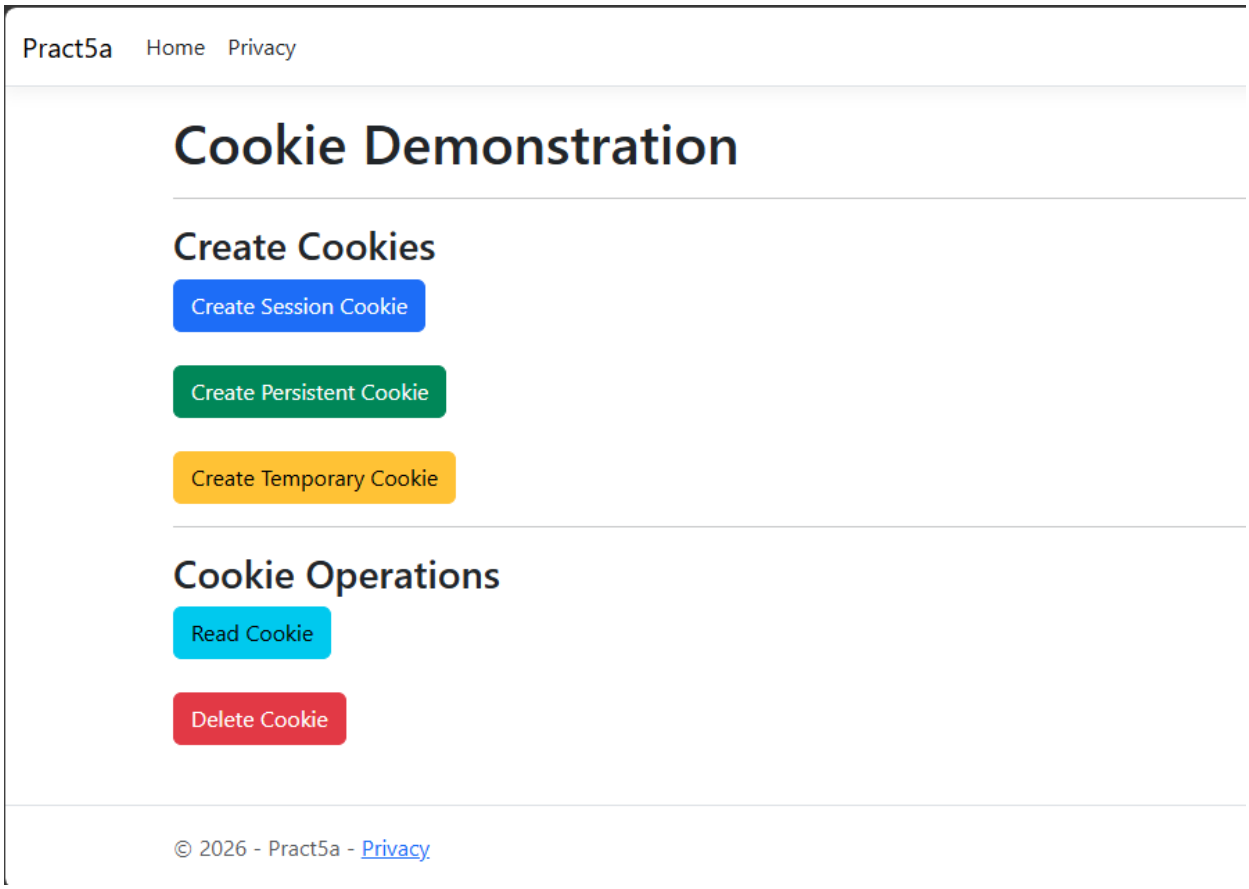
```
Ctrl + F5
```

The browser will open.

Click:

Create Session Cookie

Then:
Read Cookie
You should get:



Practical 5.b

Product Table and Insert Operation Using EF Core

Aim: To create a **Product table in SQL Server** and perform an **Insert operation using Entity Framework Core**.

Step 1: Create Project

Create another project:

```
ASP.NET Core Web App (Model-View-Controller)
```

Project name:

```
ProductEFCore
```

Framework:

```
.NET 10.0
```

Authentication:

```
None
```

Click **Create**.

Step 2: Install EF Core Packages

Go to:

```
Tools
→ NuGet Package Manager
→ Package Manager Console
```

Run:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

Then:

```
Install-Package Microsoft.EntityFrameworkCore.Tools
```

These packages allow EF Core to communicate with SQL Server and create migrations.

Step 3: Create Product Model

Right-click:

```
Models
→ Add
→ Class
```

Name:

Product.cs

Write:

```
using System.ComponentModel.DataAnnotations;

namespace ProductEFCore.Models
{
    public class Product
    {
        [Key]
        public int ProductId { get; set; }

        [Required]
        public string ProductName { get; set; } = "";
```

```
        public decimal Price { get; set; }

        public int Quantity { get; set; }
    }
}
```

Step 4: Create DbContext

Right-click the project:

```
Add
→ New Folder
```

Name:

```
Data
Right-click Data:
Add
→ Class
```

Name:

ApplicationDbContext.cs

Write:

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }
    }
}
```

Step 5: Add SQL Server Connection String

Open:

appsettings.json

Add:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertific
ate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

If you are using a normal SQL Server instance instead of LocalDB, use your SQL Server name, for example:

```
Server=.\SQLEXPRESS;
```

Step 6: Configure Program.cs

Open:

Program.cs

Write:

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlServer(
            builder.Configuration.GetConnectionString(
                "DefaultConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Product}/{action=Index}/{id?}");

app.Run();
```

Step 7: Create Database Using Migration

Open:

```
Tools
→ NuGet Package Manager
→ Package Manager Console
```

Run:

```
Add-Migration InitialCreate
```

Then:

```
Update-Database
```

EF Core will create:

```
ProductDB
   ↓
Products
```

with columns:

```
ProductId
ProductName
Price
Quantity
```

Step 8: Create Product Controller

Right-click:

```
Controllers
→ Add
→ Controller
```

Select:

```
MVC Controller - Empty
```

Name:

```
ProductController
```

Click **Add**.

Write:

```
using Microsoft.AspNetCore.Mvc;
using ProductEFCore.Data;
using ProductEFCore.Models;

namespace ProductEFCore.Controllers
{
    public class ProductController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductController(
            ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Insert(Product product)
        {
            if (ModelState.IsValid)
            {
                 context.Products.Add(product);
                 context.SaveChanges();

                ViewBag.Message =
                    "Product inserted successfully!";
            }

            return View("Index");
        }
    }
}
```

Step 9: Create View

Create folder:

```
Views
→ Product
```

Add:

Index.cshtml

Write:

```
@model ProductEFCore.Models.Product

<h2>Product Registration</h2>
```

```
<form asp-controller="Product"
      asp-action="Insert"
      method="post">

    <div>
        <label>Product Name</label>
        <input asp-for="ProductName" />
    </div>

    <br />

    <div>
        <label>Price</label>
        <input asp-for="Price" />
    </div>

    <br />

    <div>
        <label>Quantity</label>
        <input asp-for="Quantity" />
    </div>

    <br />

    <button type="submit">
        Insert Product
    </button>

</form>

<br />

@if (ViewBag.Message != null)
{
    <h3>@ViewBag.Message</h3>
}
```

Step 10: Run

Press:

```
Ctrl + F5
```

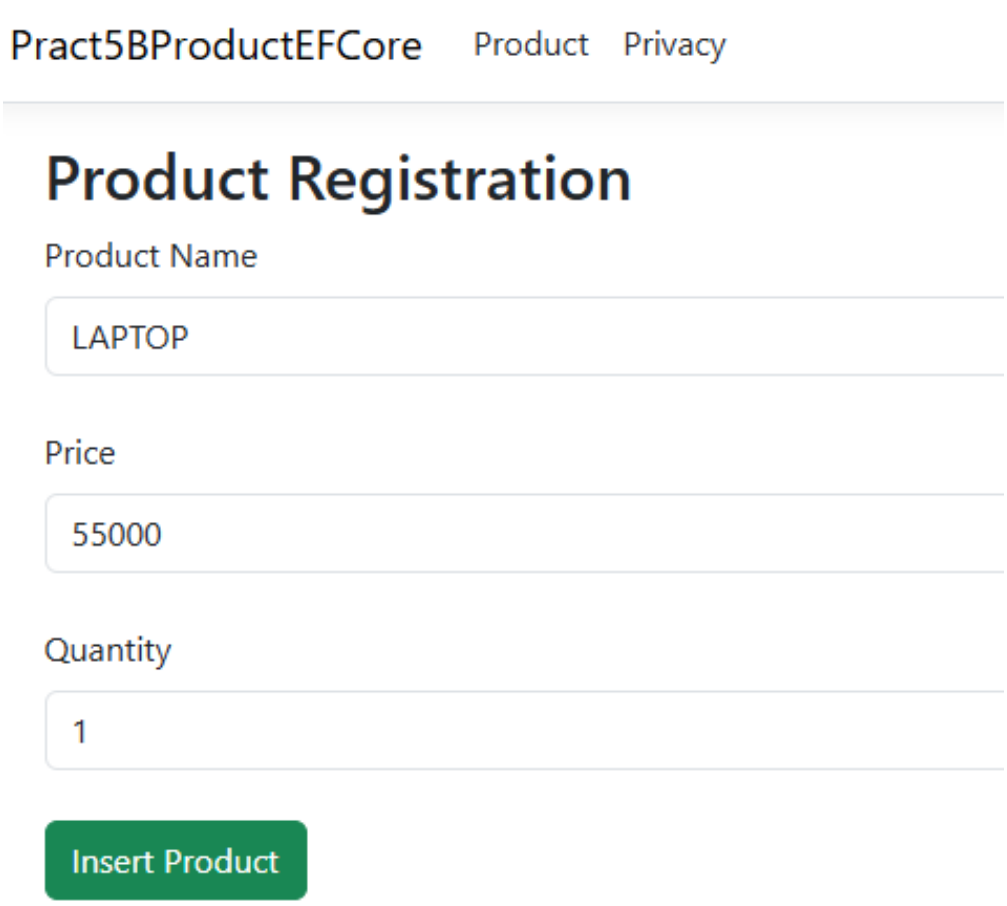
Enter:

```
Product Name: Laptop
Price: 55000
Quantity: 10
```

Click:

Insert Product

Output:


Pract5BProductEFCore
Product
Privacy
Product Registration
Product Name
LAPTOP
Price
55000
Quantity
1
Insert Product

Practical 5.c

CRUD Operations on Employee Table Using EF Core

Aim: To develop an ASP.NET Core MVC application to perform:

- Create
- Read
- Update
- Delete

operations on an **Employee** table using Entity Framework Core.

Step 1: Create Project

Create:

```
ASP.NET Core Web App (Model-View-Controller)
```

Project name:

```
EmployeeCRUD
```

Framework:

```
.NET 10.0
```

Authentication:

```
None
```

Step 2: Install EF Core

Open:

```
Tools
→ NuGet Package Manager
→ Package Manager Console
```

Run:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

Then:

```
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Step 3: Create Employee Model

Create:

```
Models → Employee.cs
```

Code:

```
using System.ComponentModel.DataAnnotations;

namespace EmployeeCRUD.Models
{
    public class Employee
    {
        [Key]
        public int EmployeeId { get; set; }

        [Required]
        public string Name { get; set; } = "";

        [Required]
        public string Department { get; set; } = "";
```

```
        public decimal Salary { get; set; }
    }
}
```

Step 4: Create DbContext

Create:

```
Data → ApplicationDbContext.cs
```

Code:

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

Step 5: Add Connection String

Open:

appsettings.json

Add:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

Step 6: Configure Program.cs

```
using EmployeeCRUD.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlServer(
            builder.Configuration.GetConnectionString(
                "DefaultConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
```

```
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Employee}/{action=Index}/{id?}");

app.Run();
```

Step 7: Create Database

Package Manager Console:

```
Add-Migration InitialCreate
```

Then:

```
Update-Database
```

SQL Server will contain:

```
EmployeeDB
   └── Employees
```

Step 8: Create Employee Controller

Right-click:

```
Controllers
→ Add
→ Controller
```

Select:

```
MVC Controller - Empty
```

Name:

```
EmployeeController
```

Replace the generated code with:

```
using EmployeeCRUD.Data;
using EmployeeCRUD.Models;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeCRUD.Controllers
{
    public class EmployeeController : Controller
    {
        private readonly ApplicationDbContext  context;

        public EmployeeController(
            ApplicationDbContext context)
        {
            _context = context;
        }

        // READ
        public IActionResult Index()
        {
            var employees = _context.Employees.ToList();

            return View(employees);
        }

        // CREATE - GET
```

```
        public IActionResult Create()
        {
            return View();
        }

        // CREATE - POST
        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Employee employee)
        {
            if (ModelState.IsValid)
            {
                 context.Employees.Add(employee);
                 context.SaveChanges();

                return RedirectToAction("Index");
            }

            return View(employee);
        }

        // UPDATE - GET
        public IActionResult Edit(int id)
        {
            var employee =
                _context.Employees.Find(id);

            if (employee == null)
            {
                return NotFound();
            }

            return View(employee);
        }

        // UPDATE - POST
        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Edit(
            int id,
            Employee employee)
        {
            if (id != employee.EmployeeId)
            {
                return NotFound();
            }

            if (ModelState.IsValid)
            {
                _context.Employees.Update(employee);
                _context.SaveChanges();

                return RedirectToAction("Index");
            }

            return View(employee);
        }

        // DELETE
        public IActionResult Delete(int id)
        {
            var employee =
                _context.Employees.Find(id);

            if (employee == null)
            {
                return NotFound();
            }

            _context.Employees.Remove(employee);
            _context.SaveChanges();

            return RedirectToAction("Index");
        }
    }
}
```

Step 9: Create Views

Create this folder:

```
Views
└── Employee
```

Create:

```
Index.cshtml
Create.cshtml
Edit.cshtml
```

Index.cshtml — Read Operation

```
@model IEnumerable<EmployeeCRUD.Models.Employee>

<h2>Employee List</h2>

<a asp-action="Create">
    Add New Employee
</a>

<br />
<br />

<table border="1" cellpadding="10">

    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Department</th>
        <th>Salary</th>
        <th>Actions</th>
    </tr>

    @foreach (var employee in Model)
    {
        <tr>
            <td>@employee.EmployeeId</td>
            <td>@employee.Name</td>
            <td>@employee.Department</td>
            <td>@employee.Salary</td>

            <td>
                <a asp-action="Edit"
                   asp-route-id="@employee.EmployeeId">
                    Edit
                </a>

                |

                <a asp-action="Delete"
                   asp-route-id="@employee.EmployeeId">
                    Delete
                </a>
            </td>
        </tr>
    }

</table>
```

Create.cshtml — Create Operation

```
@model EmployeeCRUD.Models.Employee

<h2>Add Employee</h2>

<form asp-action="Create" method="post">

    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>

    <br />
```

```
    <div>
        <label>Department</label>
        <input asp-for="Department" />
    </div>

    <br />

    <div>
        <label>Salary</label>
        <input asp-for="Salary" />
    </div>

    <br />

    <button type="submit">
        Save
    </button>

</form>
```

Edit.cshtml — Update Operation

```
@model EmployeeCRUD.Models.Employee

<h2>Edit Employee</h2>

<form asp-action="Edit" method="post">

    <input type="hidden"
           asp-for="EmployeeId" />

    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>

    <br />

    <div>
        <label>Department</label>
        <input asp-for="Department" />
    </div>

    <br />

    <div>
        <label>Salary</label>
        <input asp-for="Salary" />
    </div>

    <br />

    <button type="submit">
        Update
    </button>

</form>
```

Step 10: Test CRUD

Run:

```
Ctrl + F5
```

Create

Click:

```
Add New Employee
```

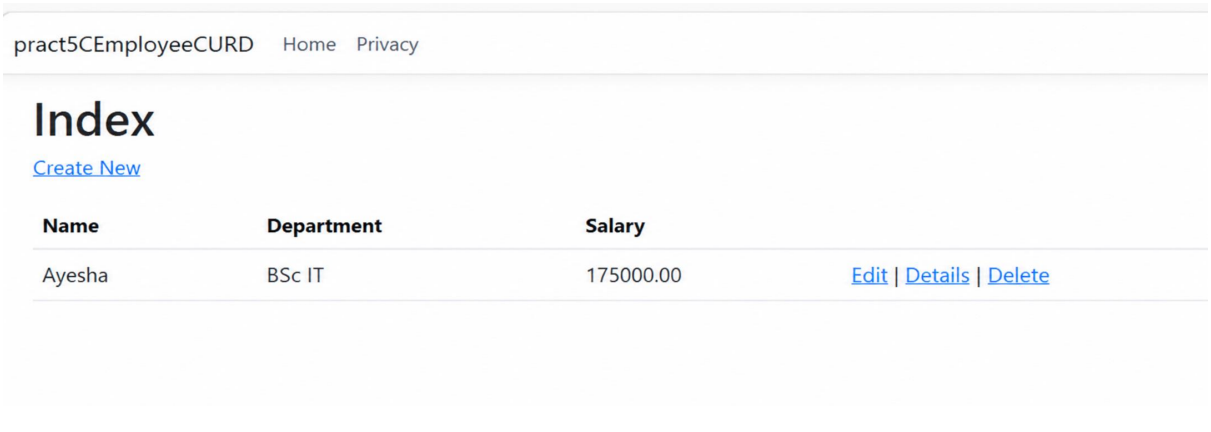
Enter:

```
Name: Ayesha
Department: BSc.IT
Salary: 175000
```

Click **Save**.

Read

Employee list will display:



Update

Click:

```
Edit
```

Change:

```
Salary: 50000
```

Click:

```
Update
```

Delete

Click:

```
Delete
```

The employee will be removed from the database.

CRUD flow

```
            Employee
               |
     ┌─────────┼─────────┐
     ↓         ↓         ↓
  CREATE     READ     UPDATE
                         |
                         ↓
                      DELETE
```

Practical 5.d

Authentication and Authorization in ASP.NET Core

Aim: To demonstrate **Authentication and Authorization** in an ASP.NET Core MVC application.

Difference

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to access?

For this practical, we'll use **ASP.NET Core Identity** so that the application provides registration/login functionality and role-based authorization.

Step 1: Create Project

Open Visual Studio 2026.

Select:

```
Create a new project
```

Choose:

```
ASP.NET Core Web App (Model-View-Controller)
```

Project name:

```
AuthDemo
```

Framework:

```
.NET 10.0
```

For Authentication type, select:

```
Individual Accounts
```

Click:

```
Create
```

This creates an application with ASP.NET Core Identity.

Step 2: Install EF Core SQL Server Package

Open:

```
Tools
→ NuGet Package Manager
→ Package Manager Console
```

Run:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

Step 3: Check Identity DbContext

The Identity template normally creates:

```
Data
└── ApplicationDbContext.cs
```

It will contain something similar to:

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace AuthDemo.Data
{
    public class ApplicationDbContext
        : IdentityDbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
    }
}
```

Step 4: Configure Database

Open:

appsettings.json

Use:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=AuthDemoDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

Step 5: Configure Identity in Program.cs

```
The Individual Accounts template already configures Identity. Your Program.cs should contain the Identity registration and database configuration.
```

A typical setup is:

```
using AuthDemo.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

var connectionString =
    builder.Configuration.GetConnectionString(
        "DefaultConnection");

builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlServer(connectionString));

builder.Services.AddDefaultIdentity<IdentityUser>(
    options =>
    {
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.AddControllersWithViews();

builder.Services.AddRazorPages();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
```

```
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();

app.Run();
```

Very important

Authentication middleware:

```
app.UseAuthentication();
```

must be before:

```
app.UseAuthorization();
```

Step 6: Create Identity Database

Open Package Manager Console.

Run:

```
Add-Migration CreateIdentityDatabase
```

Then:

```
Update-Database
```

SQL Server will create the Identity tables.

You will see tables such as:

```
AspNetUsers
AspNetRoles
AspNetUserRoles
AspNetUserClaims
AspNetUserLogins
AspNetUserTokens
```

Step 7: Run Authentication Application

Press:

```
Ctrl + F5
```

The website will open.

You should see:

```
Register
Login
```

Click **Register**.

Enter an email and password.

Example:

```
Email:
ayesha@gmail.com

Password:
Ayesha@12345
```

Click **Register**.

You have now demonstrated:

Authentication

```
Register
   ↓
Login
   ↓
Authenticated User
```

Step 8: Create Authorized Controller

Create:

```
Controllers
→ HomeController.cs
```

Add:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [Authorize]
        public IActionResult Student()
        {
            return View();
        }

        [Authorize]
        public IActionResult Dashboard()
        {
            return View();
        }
    }
}
```

The important part is:

```
[Authorize]
```

Only authenticated users can access these actions.

Step 9: Create Student View

Create:

```
Views
└── Home
    └── Student.cshtml
```

Write:

```
@{
    ViewData["Title"] = "Student";
}

<h2>Student Page</h2>
```

```
<p>
    Welcome! You are an authenticated user.
</p>
```

Step 10: Test Authorization

First, log out.

Try to open:

```
/Home/Student
```

The application will redirect the unauthenticated user to the login page.

Now:

1. Login.
2. Open /Home/Student.

The page will be accessible.

Therefore:

```
Not Logged In
      ↓
/Home/Student
      ↓
   Login Page


Logged In
    ↓
/Home/Student
    ↓
Access Granted
```

Role-Based Authorization

For a more complete practical, we can demonstrate **Admin/User roles**.

Change the Identity configuration from:

```
AddDefaultIdentity<IdentityUser>
```

to:

```
AddDefaultIdentity<IdentityUser>
```

and add roles:

```
builder.Services.AddDefaultIdentity<IdentityUser>(
    options =>
    {
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

The important addition is:

```
.AddRoles<IdentityRole>()
```

Step 11: Create Admin Role

Add this code before:

```
app.Run();
using Microsoft.AspNetCore.Identity;

using (var scope = app.Services.CreateScope())
{
    var roleManager =
```

```
        scope.ServiceProvider
        .GetRequiredService<RoleManager<IdentityRole>>();

    if (!await roleManager.RoleExistsAsync("Admin"))
    {
        await roleManager.CreateAsync(
            new IdentityRole("Admin"));
    }
}
```

Your final section will look approximately like:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();

using (var scope = app.Services.CreateScope())
{
    var roleManager =
        scope.ServiceProvider
        .GetRequiredService<RoleManager<IdentityRole>>();

    if (!await roleManager.RoleExistsAsync("Admin"))
    {
        await roleManager.CreateAsync(
            new IdentityRole("Admin"));
    }
}

app.Run();
```

Step 12: Create Admin-Only Controller

Create:

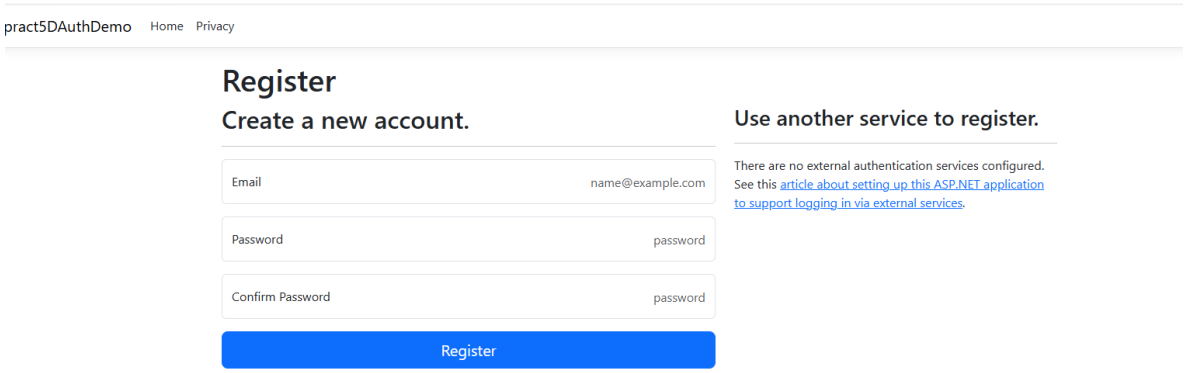
```
Controllers
→ AdminController.cs
```

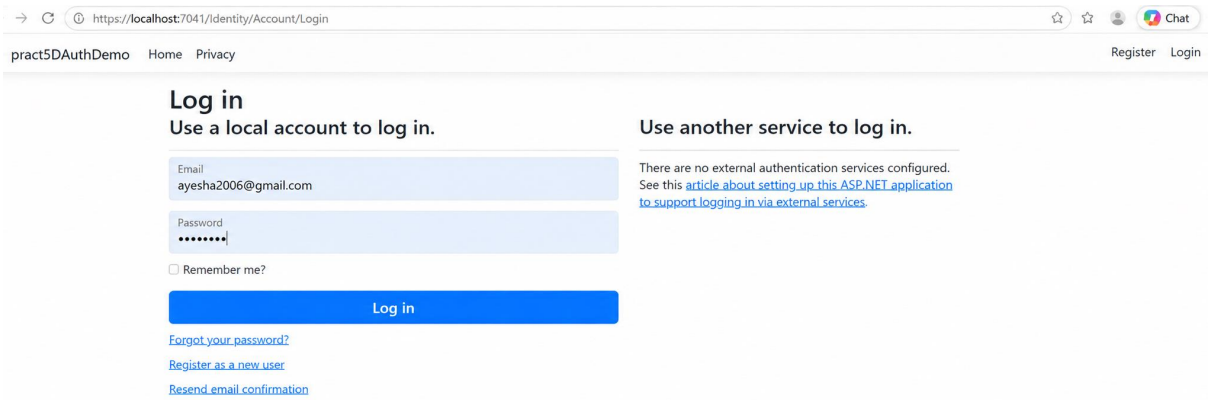
Code:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace AuthDemo.Controllers
{
    [Authorize(Roles = "Admin")]
    public class AdminController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Output:


pract5DAuthDemo
Home
Privacy
Register
Create a new account.
Email
name@example.com
Password
password
Confirm Password
password
Register
Use another service to register.
There are no external authentication services configured.
See this article about setting up this ASP.NET application to support logging in via external services.


https://localhost:7041/Identity/Account/Login
Chat
pract5DAuthDemo
Home
Privacy
Register
Login
Log in
Use a local account to log in.
Email
ayesha2006@gmail.com
Password
Remember me?
Log in
Forgot your password?
Register as a new user
Resend email confirmation
Use another service to log in.
There are no external authentication services configured.
See this article about setting up this ASP.NET application to support logging in via external services.

Practical 6

Aim: a. Create a simple website for a College Information Portal and convert it into a Progressive Web Application (PWA).

b. Create a PWA that displays a custom offline page when the user is not connected to the Internet.

Part A — College Information Portal PWA

1. Create ASP.NET Core MVC Project

Open Visual Studio 2026.

Select:

Create a new project → ASP.NET Core Web App (Model-View-Controller)

Enter:

Project Name: CollegePortalPWA

Framework: .NET 10.0

Authentication: None

Click Create.

2. Project Structure

Your project will look approximately like this:

CollegePortalPWA

```
│
├── Controllers
│   └── HomeController.cs
│
├── Models
│
├── Views
│   ├── Home
│   │   ├── Index.cshtml
│   │   ├── About.cshtml
│   │   └── Contact.cshtml
│   │
│   └── Shared
│       └──  Layout.cshtml
│
├── wwwroot
│   ├── css
│   │   └── site.css
│   │
│   ├── js
│   │   └── site.js
│   │
│   ├── images
│   │
│   ├── manifest.json
│   ├── service-worker.js
│   └── offline.html
│
└── Program.cs
```

Important: manifest.json, service-worker.js, and offline.html should be inside wwwroot.

3. Create College Home Page

Open:

Views → Home → Index.cshtml

Replace the existing code with:

```
@{
    ViewData["Title"] = "College Information Portal";
```

```
}

<div class="hero">
    <h1>College Information Portal</h1>
    <p>Welcome to our College Information Portal</p>
    <a class="btn btn-primary" asp-controller="Home" asp-action="About">
        Learn More
    </a>
</div>

<div class="container mt-5">

    <h2 class="text-center mb-4">College Information</h2>

    <div class="row">

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>🎓 Courses</h3>
                <p>
                    We offer undergraduate and postgraduate courses
                    in Science, Commerce and Management.
                </p>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>📚 Library</h3>
                <p>
                    Our library provides books, journals and
                    digital learning resources for students.
                </p>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>🏫 Campus</h3>
                <p>
                    Our campus provides modern classrooms,
                    laboratories and student facilities.
                </p>
            </div>
        </div>

    </div>

    <div class="row mt-3">

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>👨‍🏫 Faculty</h3>
                <p>
                    Experienced and supportive faculty members
                    guide students throughout their education.
                </p>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>💼 Placements</h3>
                <p>
                    Career guidance and placement opportunities
                    are provided to students.
                </p>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card p-3 mb-3">
                <h3>🎉 Events</h3>
                <p>
                    Students can participate in cultural,
                    technical and sports activities.
                </p>
            </div>
```

```
        </div>

    </div>

</div>
```

4. Create About Page

Create:

Views

→ Home

→ About.cshtml

Add:

```
@{
    ViewData["Title"] = "About College";
}

<div class="container mt-5">

    <h1>About Our College</h1>

    <p>
        Our college provides quality education and a
        student-friendly learning environment.
    </p>

    <h3>Our Vision</h3>

    <p>
        To provide quality education and develop
        responsible and skilled professionals.
    </p>

    <h3>Our Mission</h3>

    <p>
        To encourage academic excellence, innovation,
        leadership and overall development of students.
    </p>

</div>
```

5. Create Contact Page

Create:

Views

→ Home

→ Contact.cshtml

Add:

```
@{
    ViewData["Title"] = "Contact";
}

<div class="container mt-5">

    <h1>Contact Us</h1>

    <p><strong>College:</strong> ABC College</p>

    <p><strong>Address:</strong> Mumbai, Maharashtra</p>

    <p><strong>Email:</strong> info@college.edu</p>

    <p><strong>Phone:</strong> 9876543210</p>

</div>
```

6. Update HomeController

Open:

Controllers → HomeController.cs

Make sure it contains:

```
using Microsoft.AspNetCore.Mvc;

namespace CollegePortalPWA.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult About()
        {
            return View();
        }

        public IActionResult Contact()
        {
            return View();
        }
    }
}
```

7. Update Navigation Menu

Open:

Views → Shared → _Layout.cshtml

Find the navigation section and use:

```
<ul class="navbar-nav flex-grow-1">

    <li class="nav-item">
        <a class="nav-link text-dark"
           asp-controller="Home"
           asp-action="Index">
            Home
        </a>
    </li>

    <li class="nav-item">
        <a class="nav-link text-dark"
           asp-controller="Home"
           asp-action="About">
            About
        </a>
    </li>

    <li class="nav-item">
        <a class="nav-link text-dark"
           asp-controller="Home"
           asp-action="Contact">
            Contact
        </a>
    </li>

</ul>
```

8. Add CSS

Open:

wwwroot/css/site.css

Add:

```
.hero {
    background: linear-gradient(135deg, #0d6efd, #6610f2);
    color: white;
```

```
    text-align: center;
    padding: 80px 20px;
}

.hero h1 {
    font-size: 45px;
    font-weight: bold;
}

.hero p {
    font-size: 20px;
}

.card {
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
    min-height: 180px;
}

.card h3 {
    margin-bottom: 15px;
}
```

Part B — Convert Website into PWA

A Progressive Web Application requires mainly:

1. Web App Manifest
2. Service Worker
3. HTTPS
4. Responsive website
5. Offline support

We will create these files manually.

9. Create manifest.json

Inside:

wwwroot

Right-click → Add → New Item

Create:

manifest.json

Add:

```
{
  "name": "College Information Portal",
  "short name": "College Portal",
  "start url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme color": "#0d6efd",
  "description": "Progressive Web Application for College Information Portal",
  "icons": [
    {
      "src": "/images/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/images/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

10. Create PWA Icons

Create an images folder:

wwwroot

```
    └── images
```

Add:

icon-192.png

icon-512.png

For your practical, you can use a college logo or create simple square PNG icons.

The structure should be:

wwwroot

```
│
├── images
│   ├── icon-192.png
│   └── icon-512.png
│
├── manifest.json
└── service-worker.js
```

11. Add Manifest to Layout

Open:

Views → Shared → _Layout.cshtml

Inside <head>, add:

```
<link rel="manifest" href="~/manifest.json" />

<meta name="theme-color" content="#0d6efd" />

<meta name="mobile-web-app-capable" content="yes" />
```

For example:

```
<head>

    <meta charset="utf-8" />

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0" />

    <title>@ViewData["Title"] - College Portal</title>

    <link rel="manifest" href="~/manifest.json" />

    <meta name="theme-color" content="#0d6efd" />

    <meta name="mobile-web-app-capable" content="yes" />

    <link rel="stylesheet"
          href="~/lib/bootstrap/dist/css/bootstrap.min.css" />

    <link rel="stylesheet"
          href="~/css/site.css"
          asp-append-version="true" />

</head>
```

12. Create Service Worker

Inside:

wwwroot

Create:

service-worker.js

Add:

```
const CACHE_NAME = "college-portal-v1";

const urlsToCache = [
```

```
    "/",
    "/Home/About",
    "/Home/Contact",
    "/css/site.css",
    "/offline.html",
    "/manifest.json"
];

self.addEventListener("install", event => {

    event.waitUntil(
        caches.open(CACHE NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );

    self.skipWaiting();
});


self.addEventListener("activate", event => {

    event.waitUntil(
        caches.keys().then(cacheNames => {

            return Promise.all(
                cacheNames
                    .filter(name => name !== CACHE NAME)
                    .map(name => caches.delete(name))
            );

        })
    );

    self.clients.claim();
});


self.addEventListener("fetch", event => {

    event.respondWith(

        fetch(event.request)
            .catch(() => {

                return caches.match(event.request)
                    .then(response => {

                        if (response) {
                            return response;
                        }

                        return caches.match("/offline.html");

                    });

            })

    );

});
```

13. Create Custom Offline Page

This is especially important for Part B.

Inside:

wwwroot

create:

offline.html

Add:

```
<!DOCTYPE html>
```

```html
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Offline - College Portal</title>

    <style>

        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background: #f5f5f5;
            padding: 60px 20px;
        }

        .offline-box {
            max-width: 500px;
            margin: auto;
            background: white;
            padding: 40px;
            border-radius: 15px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.15);
        }

        h1 {
            color: #0d6efd;
        }

        .icon {
            font-size: 70px;
        }

        button {
            padding: 12px 25px;
            border: none;
            border-radius: 6px;
            background: #0d6efd;
            color: white;
            cursor: pointer;
        }

    </style>

</head>

<body>

    <div class="offline-box">

        <div class="icon">📡</div>

        <h1>You are Offline</h1>

        <p>
            The College Information Portal is currently
            unable to connect to the Internet.
        </p>

        <p>
            Please check your Internet connection
            and try again.
        </p>

        <button onclick="location.reload()">
            Try Again
        </button>

    </div>

</body>

</html>
```

14. Register Service Worker

Open:

wwwroot/js/site.js

Add:

```
if ("serviceWorker" in navigator) {

    window.addEventListener("load", () => {

        navigator.serviceWorker
            .register("/service-worker.js")
            .then(registration => {

                console.log(
                    "Service Worker registered:",
                    registration.scope
                );

            })
            .catch(error => {

                console.log(
                    "Service Worker registration failed:",
                    error
                );

            });

    });

}
```

15. Check Program.cs

For an ASP.NET Core MVC application, make sure static files are being served.

For a typical .NET 10 project:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Important

If your .NET 10 project template already contains:

```
app.MapStaticAssets();
```

you can keep the template's static-asset configuration. The important requirement is that files under wwwroot such as:

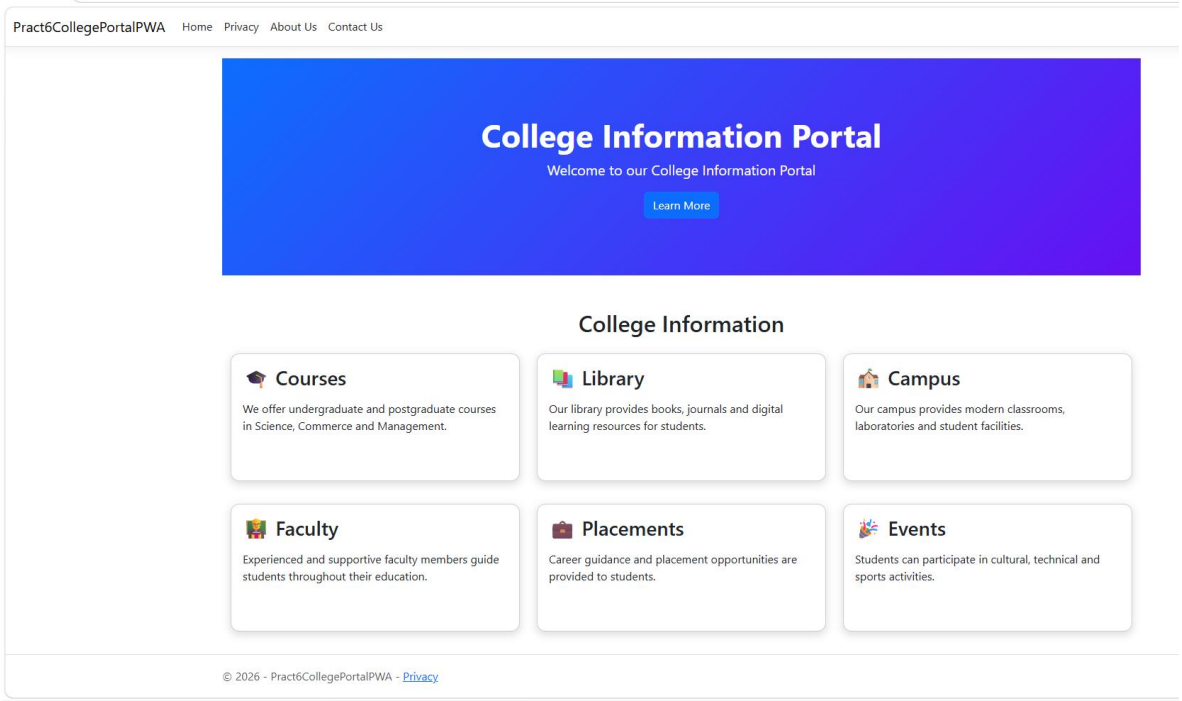
manifest.json

service-worker.js
offline.html
must be accessible from the browser.

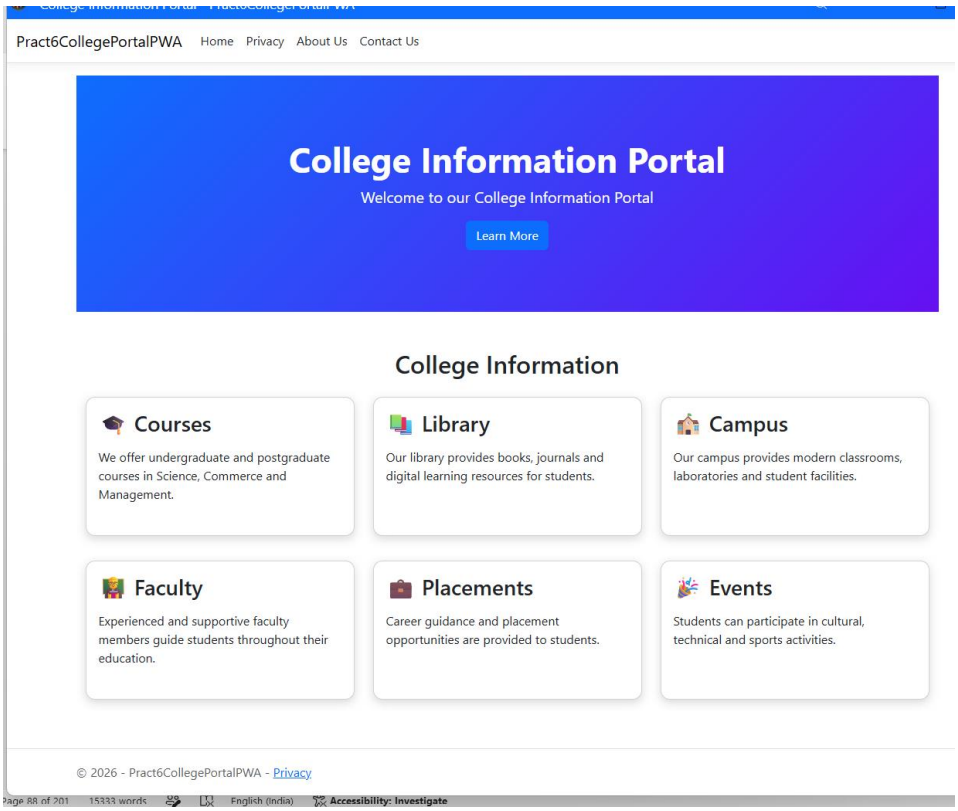
16. Run the Application

Press:
Ctrl + F5
or:
Ctrl + F5
The website should display:
College Information Portal

Welcome to our College Information Portal
Part (a)



Part (b)
Offline page after downloading collage portal app like application


Pract6CollegePortalPWA
Home
Privacy
About Us
Contact Us
College Information Portal
Welcome to our College Information Portal
Learn More
College Information
Courses
We offer undergraduate and postgraduate courses in Science, Commerce and Management.
Library
Our library provides books, journals and digital learning resources for students.
Campus
Our campus provides modern classrooms, laboratories and student facilities.
Faculty
Experienced and supportive faculty members guide students throughout their education.
Placements
Career guidance and placement opportunities are provided to students.
Events
Students can participate in cultural, technical and sports activities.
© 2026 - Pract6CollegePortalPWA - Privacy

Practical 7

7(a). Develop a PWA that works in offline mode using Service Worker.
7(b). Create a Student Record Management PWA.

We will build both together as one ASP.NET Core MVC PWA.

Practical 7.b: Student Record Management PWA

Aim: To develop a Progressive Web Application (PWA) using ASP.NET Core MVC that:

1. Works in offline mode using a Service Worker.
2. Provides a Student Record Management System.
3. Allows the user to:
    - Add students
    - View students
    - Search students
    - Delete students
4. Stores student records in browser Local Storage, so records remain available even when the internet is unavailable.
5. Can be installed as a PWA.

1. Technologies Used

- ASP.NET Core MVC
- C#
- HTML5
- CSS3
- JavaScript
- Service Worker
- Web App Manifest
- Local Storage
- Visual Studio 2026
- .NET 10

2. Create the Project

Open Visual Studio 2026.

Select:

Create a new project

Choose:

ASP.NET Core Web App (Model-View-Controller)

Project name:

StudentRecordPWA

Framework:

.NET 10.0

Authentication:

None

Click Create.

3. Project Structure

After creating the project, create the following structure:

StudentRecordPWA

```
│
├── Controllers
│   └── HomeController.cs
│
├── Models
│   └── Student.cs
│
├── Views
│   ├── Home
│   │   └── Index.cshtml
│   │
│   └── Shared
│       └── _Layout.cshtml
│
├── wwwroot
│   │
│   ├── css
│   │   └── site.css
│   │
│   ├── js
│   │   └── site.js
│   │
│   ├── images
│   │   ├── icon-192.png
│   │   └── icon-512.png
│   │
│   ├── manifest.json
│   ├── service-worker.js
│   └── offline.html
│
└── Program.cs
```

The important PWA files are:
manifest.json
service-worker.js
offline.html
site.js

4. Create Student Model

Go to:
Models

```
    → Student.cs
```

Replace the code with:

```
namespace StudentRecordPWA.Models
{
    public class Student
    {
        public int Id { get; set; }

        public string Name { get; set; } = "";

        public string Course { get; set; } = "";

        public int Age { get; set; }

        public string Email { get; set; } = "";
    }
}
```

Explanation
The Student class represents one student record.

Property	Purpose
Id	Student ID
Name	Student name
Course	Course such as B.Sc. IT
Age	Student age
Email	Student email

For this practical, we are using Local Storage instead of SQL Server, because we want the student records to work offline.

5. Create HomeController

Open:

Controllers/HomeController.cs

Use:

```
using Microsoft.AspNetCore.Mvc;

namespace StudentRecordPWA.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Explanation

The controller contains the Index() action.

```
public IActionResult Index()
```

When the user visits:

/

ASP.NET Core executes:

HomeController

```
        ↓
Index()
        ↓
```

Views/Home/Index.cshtml

6. Create Student Management Page

Open:

Views/Home/Index.cshtml

Replace the existing code with:

```
@{
    ViewData["Title"] = "Student Record Management";
}

<div class="container mt-4">

    <div class="text-center mb-4">
        <h1>Student Record Management</h1>
        <p class="lead">
```

```
            Manage student records even when you are offline.
        </p>

        <span id="connectionStatus" class="badge bg-success">
            Online
        </span>
    </div>

    <!-- Add Student -->
    <div class="card shadow mb-4">
        <div class="card-header">
            <h4>Add Student</h4>
        </div>

        <div class="card-body">

            <form id="studentForm">

                <div class="row">

                    <div class="col-md-6 mb-3">
                        <label class="form-label">
                            Student Name
                        </label>

                        <input type="text"
                               id="name"
                               class="form-control"
                               placeholder="Enter student name"
                               required />
                    </div>

                    <div class="col-md-6 mb-3">
                        <label class="form-label">
                            Course
                        </label>

                        <select id="course"
                                class="form-control"
                                required>

                            <option value="">
                                Select Course
                            </option>

                            <option value="B.Sc. IT">
                                B.Sc. IT
                            </option>

                            <option value="B.Sc. CS">
                                B.Sc. CS
                            </option>

                            <option value="BCA">
                                BCA
                            </option>

                            <option value="B.Com">
                                B.Com
                            </option>

                        </select>
                    </div>

                    <div class="col-md-6 mb-3">
                        <label class="form-label">
                            Age
                        </label>

                        <input type="number"
                               id="age"
                               class="form-control"
                               required />
                    </div>

                    <div class="col-md-6 mb-3">
                        <label class="form-label">
                            Email
                        </label>
```

```
                    <input type="email"
                           id="email"
                           class="form-control"
                           placeholder="Enter email"
                           required />
                </div>

            </div>

            <button type="submit"
                    class="btn btn-primary">
                Add Student
            </button>

            <button type="reset"
                    class="btn btn-secondary">
                Clear
            </button>

        </form>

    </div>
</div>


<!-- Search -->
<div class="card shadow mb-4">

    <div class="card-body">

        <label class="form-label">
            Search Student
        </label>

        <input type="text"
               id="searchStudent"
               class="form-control"
               placeholder="Search by name or course..." />

    </div>

</div>


<!-- Student Records -->
<div class="card shadow">

    <div class="card-header">
        <h4>Student Records</h4>
    </div>

    <div class="card-body">

        <div class="table-responsive">

            <table class="table table-bordered table-striped">

                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Course</th>
                        <th>Age</th>
                        <th>Email</th>
                        <th>Action</th>
                    </tr>
                </thead>

                <tbody id="studentTableBody">
                </tbody>

            </table>

        </div>

    </div>

</div>
```

```
</div>
```

7. Create JavaScript

Open:

wwwroot/js/site.js

Replace the code with:

```
// ====================================
// Student Record Management
// ====================================

let students = JSON.parse(
    localStorage.getItem("students")
) || [];
```

// Display students when page loads

```
document.addEventListener("DOMContentLoaded", function () {

    displayStudents(students);

    updateConnectionStatus();

    const form = document.getElementById("studentForm");

    if (form) {
        form.addEventListener("submit", addStudent);
    }

    const searchBox =
        document.getElementById("searchStudent");

    if (searchBox) {
        searchBox.addEventListener("input", searchStudents);
    }
});


// ====================================
// Add Student
// ====================================

function addStudent(event) {

    event.preventDefault();

    const name =
        document.getElementById("name").value;

    const course =
        document.getElementById("course").value;

    const age =
        document.getElementById("age").value;

    const email =
        document.getElementById("email").value;


    const student = {

        id: students.length > 0
            ? Math.max(...students.map(s => s.id)) + 1
            : 1,

        name: name,

        course: course,

        age: age,

        email: email
    };
```

```
    students.push(student);


    // Save to Local Storage
    localStorage.setItem(
        "students",
        JSON.stringify(students)
    );


    displayStudents(students);


    // Clear form
    document.getElementById("studentForm").reset();


    alert("Student added successfully!");
}


// ====================================
// Display Students
// ====================================

function displayStudents(studentList) {

    const tableBody =
        document.getElementById("studentTableBody");

    tableBody.innerHTML = "";


    if (studentList.length === 0) {

        tableBody.innerHTML = `
            <tr>
                <td colspan="6"
                    class="text-center">
                    No student records found.
                </td>
            </tr>
        `;

        return;
    }


    studentList.forEach(student => {

        const row = `

            <tr>

                <td>${student.id}</td>

                <td>${student.name}</td>

                <td>${student.course}</td>

                <td>${student.age}</td>

                <td>${student.email}</td>

                <td>

                    <button
                        class="btn btn-danger btn-sm"
                        onclick="deleteStudent(${student.id})">

                        Delete

                    </button>

                </td>

            </tr>
```

```
        `;

        tableBody.innerHTML += row;

    });
}


// ====================================
// Delete Student
// ====================================

function deleteStudent(id) {

    const confirmDelete =
        confirm("Are you sure you want to delete this student?");

    if (!confirmDelete) {
        return;
    }


    students = students.filter(
        student => student.id !== id
    );


    localStorage.setItem(
        "students",
        JSON.stringify(students)
    );


    displayStudents(students);
}


// ====================================
// Search Students
// ====================================

function searchStudents() {

    const search =
        document
            .getElementById("searchStudent")
            .value
            .toLowerCase();


    const filteredStudents =
        students.filter(student =>

            student.name
                .toLowerCase()
                .includes(search)

            ||

            student.course
                .toLowerCase()
                .includes(search)

        );


    displayStudents(filteredStudents);
}


// ====================================
// Online / Offline Status
// ====================================

function updateConnectionStatus() {

    const status =
        document.getElementById("connectionStatus");
```

```
    if (!status) {
        return;
    }


    if (navigator.onLine) {

        status.textContent = "Online";

        status.className =
            "badge bg-success";

    }
    else {

        status.textContent = "Offline";

        status.className =
            "badge bg-danger";

    }
}


window.addEventListener(
    "online",
    updateConnectionStatus
);


window.addEventListener(
    "offline",
    updateConnectionStatus
);


// ====================================
// Service Worker Registration
// ====================================

if ("serviceWorker" in navigator) {

    window.addEventListener("load", function () {

        navigator.serviceWorker
            .register("/service-worker.js")

            .then(function (registration) {

                console.log(
                    "Service Worker registered successfully:",
                    registration.scope
                );

            })

            .catch(function (error) {

                console.error(
                    "Service Worker registration failed:",
                    error
                );

            });

    });

}
```

8. What is Local Storage?

This is important for the practical.

Normally, an ASP.NET Core application might save student records in:

ASP.NET Core

```
      ↓
Controller
      ↓
Entity Framework Core
      ↓
```

SQL Server

But SQL Server requires the application/server to be available.

For this PWA practical, we use:

Browser

↓

Local Storage

↓

Student Records

For example:

```
localStorage.setItem(
    "students",
    JSON.stringify(students)
);
```

The data is stored inside the browser.

To retrieve it:

```
localStorage.getItem("students")
```

This allows student records to remain available when the device is offline.

9. Create Web App Manifest

Inside:

wwwroot

create:

manifest.json

Add:

```
{
  "name": "Student Record Management PWA",
  "short_name": "Student PWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "description": "Offline Student Record Management Progressive Web Application",
  "icons": [
    {
      "src": "/images/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/images/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

10. Manifest Explanation

name

"name": "Student Record Management PWA"

Full name of the application.

short_name

"short_name": "Student PWA"

Short name used when the application is installed.

start_url

"start_url": "/"

The page that opens when the PWA starts.

display

"display": "standalone"

Makes the application look more like a native application.

Instead of opening like a normal browser page, the installed PWA can open in its own application window.

theme_color

"theme_color": "#0d6efd"

Defines the application's theme color.

icons

The PWA needs icons such as:

icon-192.png

icon-512.png

Place them inside:

wwwroot/images/

You can use any suitable Student/College logo.

11. Modify _Layout.cshtml

Open:

Views/Shared/_Layout.cshtml

Inside the <head> section, add:

```
<link rel="manifest" href="~/manifest.json" />

<meta name="theme-color" content="#0d6efd" />

<meta name="mobile-web-app-capable"
      content="yes" />
```

Make sure the JavaScript is included near the bottom of the page:

```
<script src="~/js/site.js"
        asp-append-version="true"></script>
```

12. Create Service Worker

This is the most important part of the PWA practical.

Create:

wwwroot/service-worker.js

Add:

```
const CACHE_NAME = "student-record-pwa-v1";

const urlsToCache = [
    "/",
    "/Home/Index",
    "/css/site.css",
    "/js/site.js",
```

```
    "/manifest.json",
    "/offline.html"
];


// ====================================
// INSTALL EVENT
// ====================================

self.addEventListener("install", event => {

    event.waitUntil(

        caches.open(CACHE NAME)
            .then(cache => {

                return cache.addAll(urlsToCache);

            })

    );

    self.skipWaiting();
});


// ====================================
// ACTIVATE EVENT
// ====================================

self.addEventListener("activate", event => {

    event.waitUntil(

        caches.keys()
            .then(cacheNames => {

                return Promise.all(

                    cacheNames
                        .filter(
                            name => name !== CACHE_NAME
                        )

                        .map(
                            name => caches.delete(name)
                        )

                );

            })

    );

    self.clients.claim();
});


// ====================================
// FETCH EVENT
// ====================================

self.addEventListener("fetch", event => {

    event.respondWith(

        fetch(event.request)

            .then(response => {

                return response;

            })

            .catch(() => {

                return caches.match(event.request)

                    .then(response => {
```

```
                        if (response) {

                            return response;

                        }

                        return caches.match(
                            "/offline.html"
                        );

                    });

            })

    );

});
```

13. Understanding the Service Worker

The Service Worker sits between the web application and network.

```
              Internet
                 ↑
                 ↓
          Service Worker
            ↙          ↘
       Network        Cache
          ↓             ↓
             Browser
```

When online:

Browser

↓

Service Worker

↓

Internet

↓

ASP.NET Core Application

When offline:

Browser

↓

Service Worker

↓

Cache

↓

Offline Page

14. Service Worker install

This code:

```
self.addEventListener("install", event => {
```

runs when the Service Worker is installed.

Then:

```
caches.open(CACHE_NAME)
```

creates/opens a cache.

For example:
student-record-pwa-v1
Then:

```
cache.addAll(urlsToCache)
```

stores important files in the cache.

15. Why Are We Caching These Files?

We have:

```
const urlsToCache = [
    "/",
    "/Home/Index",
    "/css/site.css",
    "/js/site.js",
    "/manifest.json",
    "/offline.html"
];
```

These resources can then be available without accessing the internet.
For example:
Index page
CSS
JavaScript
Manifest
Offline page
16. activate Event

```
self.addEventListener("activate", event => {
```

This runs after the Service Worker has been installed.
It is commonly used to remove old caches.
For example:
student-record-pwa-v1
student-record-pwa-v2
student-record-pwa-v3
When version 3 is active, old caches can be removed.
17. fetch Event
This is what provides the offline functionality.

```
self.addEventListener("fetch", event => {
```

It is triggered when the browser requests a resource.
The Service Worker first tries:

```
fetch(event.request)
```

If the internet works, the requested resource is returned.
If the network fails:

```
.catch(() => {
```

the Service Worker searches the cache:

```
caches.match(event.request)
```

If the requested page is not available in the cache:

```
return caches.match("/offline.html");
```

the custom offline page is returned.

18. Create Custom Offline Page

Inside:

wwwroot

create:

offline.html

Use:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Offline - Student PWA</title>

    <style>

        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 100px;
            padding: 20px;
        }

        .offline-box {
            max-width: 500px;
            margin: auto;
            padding: 40px;
            border-radius: 15px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.15);
        }

        h1 {
            font-size: 40px;
        }

        p {
            font-size: 18px;
        }

        button {
            padding: 12px 25px;
            border: none;
            border-radius: 8px;
            cursor: pointer;
            font-size: 16px;
        }

    </style>

</head>

<body>

    <div class="offline-box">

        <h1>📴</h1>

        <h1>You Are Offline</h1>

        <p>
            The Student Record Management
```

```
            application cannot connect to the internet.
        </p>

        <p>
            Please check your internet connection
            and try again.
        </p>

        <button onclick="location.reload()">
            Try Again
        </button>

    </div>

</body>

</html>
```

19. ASP.NET Core Program.cs

Open:

Program.cs

Use:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Important

This line:

```
app.UseStaticFiles();
```

allows ASP.NET Core to serve files from:

wwwroot

Therefore:

wwwroot/offline.html

can be accessed as:

/offline.html

Similarly:

wwwroot/service-worker.js

becomes:

/service-worker.js

20. Add CSS

Open:

wwwroot/css/site.css

You can use:

```
body {
    background-color: #f5f7fa;
}

.card {
    border-radius: 12px;
}

.card-header {
    font-weight: bold;
}

h1 {
    font-weight: bold;
}

table {
    background-color: white;
}
```

21. How Student Records Work

Suppose we enter:

Name: Ayesha Sarguru

Course: B.Sc. IT

Age: 20

Email: ayesha@gmail.com

JavaScript creates:

```
const student = {
    id: 1,
    name: "Ayesha Sarguru",
    course: "B.Sc. IT",
    age: 20,
    email: "ayesha@gmail.com"
};
```

It is added to:

```
students.push(student);
```

Then stored in Local Storage:

```
localStorage.setItem(
    "students",
    JSON.stringify(students)
);
```

The browser now contains the student data.

22. Why Does It Work Offline?

There are actually two different offline technologies working together.

1. Service Worker

Provides the application files:

HTML

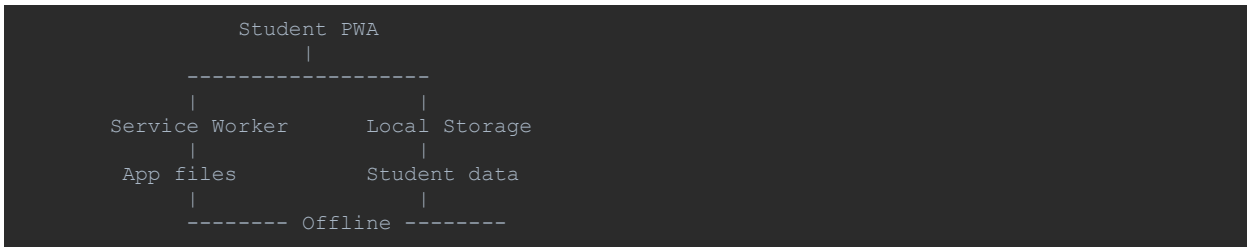
CSS

JavaScript

Offline page
from the cache.

2. Local Storage

Provides the student data:
Student ID
Name
Course
Age
Email
from the browser.
Therefore:



This is a very important point for your viva.

23. How to Test the PWA

Run the project using:
Ctrl + F5
or:
F5
Open the application.
For example:
https://localhost:xxxx/
Add some student records.
Example:

ID	Name	Course	Age	Email
1	Ayesha Sarguru	B.Sc. IT	21	ayesha@gmail.com
2	Mantasha Shaikh	BCA	20	mantasha@gmail.com
3	Saamiya Shah	B.Sc. CS	20	saamiya@gmail.com

24. Check Service Worker

Open Chrome:
F12
Go to:
Application
Then:
Service Workers
You should see something similar to:

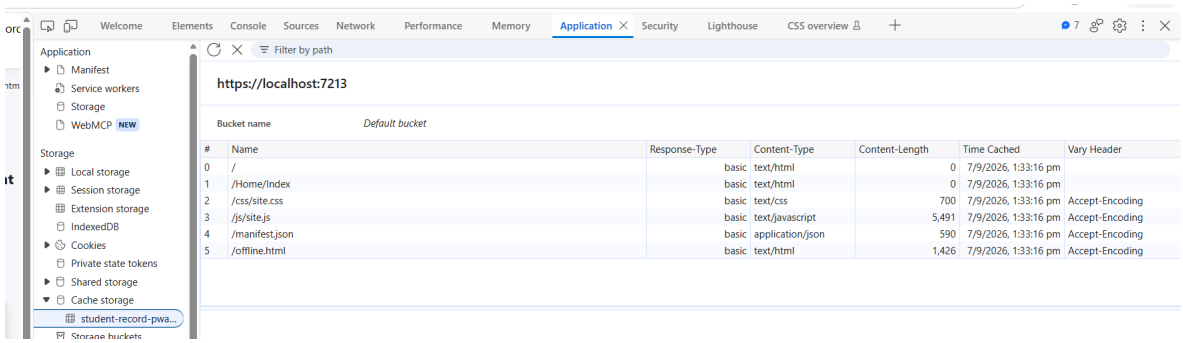
service-worker.js
Status:
activated and is running

25. Check Cache

Go to:
Application

↓

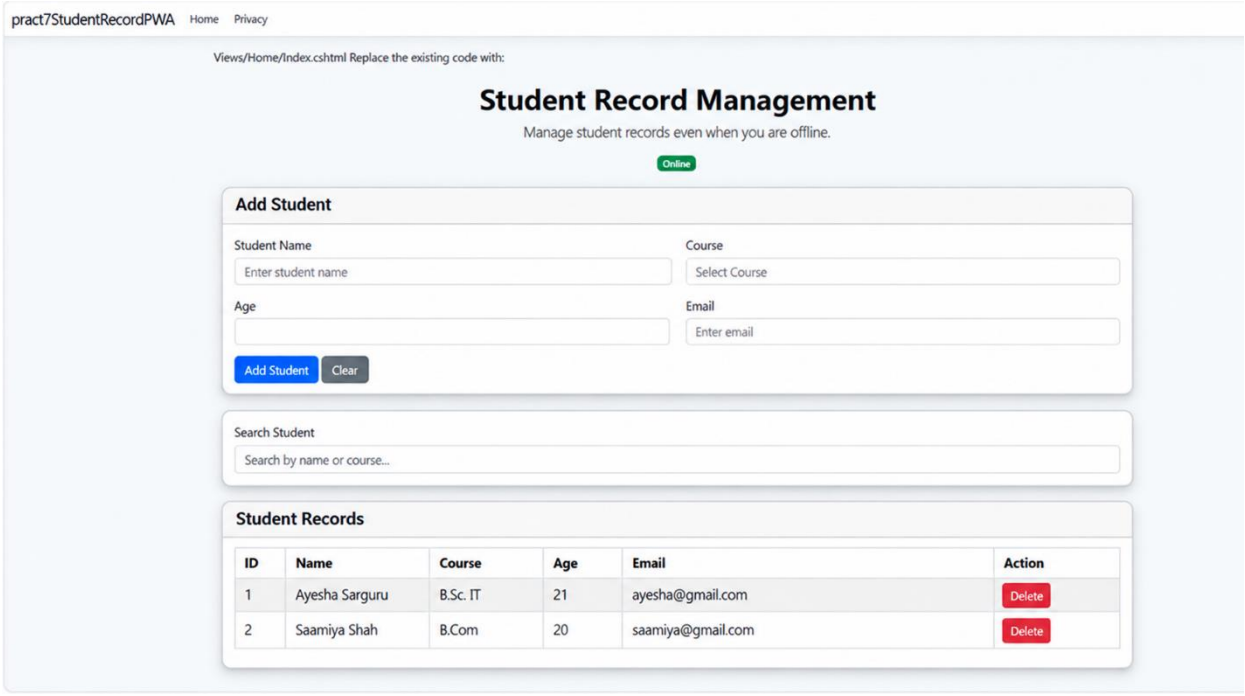
Cache Storage
You should see:



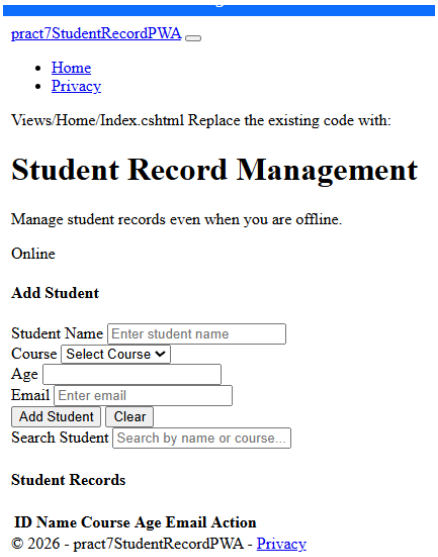
---

26. Test Offline Mode

First open the application while online.



27. Test Custom Offline Page



Practical 8

PWA Starting HTML Page and Student Web API

Aim: 8(a). Create an HTML page that would be the starting point of the application. The HTML will contain a link to manifest.json. 8(b). Create a simple Web API that returns Student data in JSON format.

Technologies Used

- ASP.NET Core MVC
- C#
- HTML5
- CSS3
- JavaScript
- Web API
- JSON
- PWA Manifest
- .NET 10.0
- Visual Studio 2026

1. Create the ASP.NET Core Project

Open Visual Studio 2026 → Create a new project → select ASP.NET Core Web App (Model-View-Controller). Project name: StudentPWAWebAPI. Framework: .NET 10.0. Authentication: None.

2. Project Structure

```
StudentPWAWebAPI
│
├── Controllers
│   ├── HomeController.cs
│   └── StudentController.cs
│
├── Models
│   └── Student.cs
│
├── Views
│   └── Shared
│       └── _Layout.cshtml
│
├── wwwroot
│   ├── css
│   │   └── style.css
│   ├── js
│   │   └── site.js
│   ├── images
│   │   ├── icon-192.png
│   │   └── icon-512.png
│   ├── index.html
│   └── manifest.json
│
└── Program.cs
```

Practical 8.a

Starting HTML Page

3. Create index.html

Inside wwwroot, create index.html and add:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Student PWA</title>

    <!-- Link Manifest File -->
    <link rel="manifest" href="manifest.json">

    <!-- Theme Color -->
    <meta name="theme-color" content="#0d6efd">

    <!-- CSS -->
    <link rel="stylesheet" href="css/style.css">
</head>

<body>

    <div class="container">

        <h1>Student Record Management</h1>

        <p>
            Welcome to Student Progressive Web Application.
        </p>

        <button onclick="loadStudents()">
            Load Students
        </button>

        <div id="studentList"></div>

    </div>

    <script src="js/site.js"></script>

</body>
</html>
```

4. Explanation of index.html

Element	Purpose
<!DOCTYPE html>	Declares that the document uses HTML5.
<html lang="en">	Root element of the HTML document; lang specifies English.
<meta charset="UTF-8">	Sets UTF-8 character encoding.
viewport meta tag	Makes the page responsive on mobile devices.
<title>Student PWA</title>	Sets the browser page title.
<link rel="manifest" href="manifest.json">	Links the starting HTML page to the PWA Web App Manifest.
theme-color	Specifies the application's theme color.
style.css	Provides the visual styling of the page.

Element	Purpose
site.js	Provides JavaScript functionality and calls the Web API.

5. Create manifest.json

Inside wwwroot, create manifest.json:

```
{
  "name": "Student Record Management",
  "short_name": "Student PWA",
  "start_url": "index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "description": "Student Record Management Progressive Web Application",
  "icons": [
    {
      "src": "images/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "images/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

6. Manifest Properties

Property	Purpose
name	Full application name.
short_name	Short name used when the PWA is installed.
start_url	Page opened when the PWA starts.
display	standalone makes the PWA open like an application.
background_color	Background color during application startup.
theme_color	Theme color used by the browser/PWA.
description	Description of the application.
icons	Application icons used by the PWA.

7. Create CSS

Create wwwroot/css/style.css:

```
body {
    font-family: Arial;
    background-color: #f4f6f9;
}

.container {
    width: 80%;
    margin: auto;
    margin-top: 40px;
    text-align: center;
}

h1 {
    color: #0d6efd;
}
```

```
button {
    padding: 10px 20px;
    background-color: #0d6efd;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

table {
    width: 100%;
    margin-top: 20px;
    border-collapse: collapse;
}

th, td {
    border: 1px solid gray;
    padding: 10px;
}

th {
    background-color: #0d6efd;
    color: white;
}
```

8. Program.cs Configurationv

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseHttpsRedirection();

app.UseDefaultFiles();
app.UseStaticFiles();

app.UseRouting();

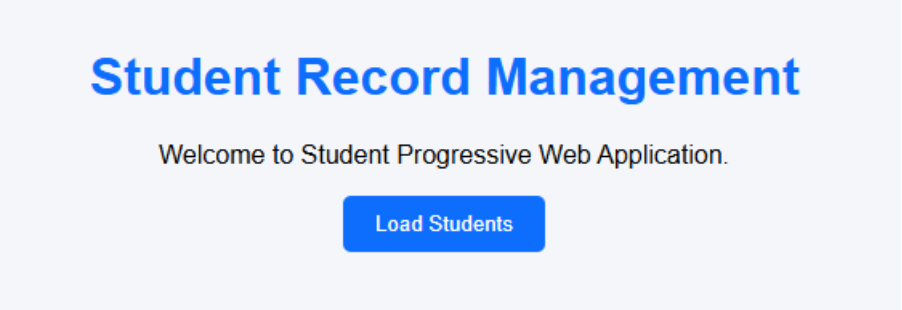
app.UseAuthorization();

app.MapControllers();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

UseDefaultFiles() allows index.html to act as a default document. UseStaticFiles() serves files from wwwroot. MapControllers() enables attribute-routed Web API controllers.



Practical 8.b

Student Web API

9. Create Student Model

```
namespace StudentPWAWebAPI.Models
{
    public class Student
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Course { get; set; }

        public int Age { get; set; }

        public string Email { get; set; }
    }
}
```

10. Create StudentController

```
using Microsoft.AspNetCore.Mvc;
using StudentPWAWebAPI.Models;

namespace StudentPWAWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetStudents()
        {
            List<Student> students = new List<Student>()
            {
                new Student
                {
                    Id = 1,
                    Name = "Ayesha Sarguru",
                    Course = "B.Sc. IT",
                    Age = 21,
                    Email = "Ayesha@gmail.com"
                },

                new Student
                {
                    Id = 2,
                    Name = "Nitin yadav",
                    Course = "BCA",
                    Age = 20,
                    Email = "Nitin@gmail.com"
                },

                new Student
                {
                    Id = 3,
                    Name = "Milan khadka",
                    Course = "B.Sc. CS",
                    Age = 20,
                    Email = "milan@gmail.com"
                }
            };

            return Ok(students);
        }
    }
}
```

11. Explanation of StudentController

Code	Explanation
[ApiController]	Marks the class as a Web API controller.

Code	Explanation
[Route("api/[controller]")]	Creates the route /api/student because the controller is StudentController.
ControllerBase	Base class commonly used for Web API controllers.
[HttpGet]	Specifies that the method handles HTTP GET requests.
IActionResult	Represents the HTTP response returned by the method.
return Ok(students)	Returns HTTP 200 OK with the student list serialized as JSON.

12. JSON Response

```
[
  new Student
                {
                    Id = 1,
                    Name = "Ayesha Sarguru",
                    Course = "B.Sc. IT",
                    Age = 21,
                    Email = "Ayesha@gmail.com"
                },

                new Student
                {
                    Id = 2,
                    Name = "Nitin yadav",
                    Course = "BCA",
                    Age = 20,
                    Email = "Nitin@gmail.com"
                },

                new Student
                {
                    Id = 3,
                    Name = "Milan khadka",
                    Course = "B.Sc. CS",
                    Age = 20,
                    Email = "milan@gmail.com"
                }
            };]
```

The API endpoint is: https://localhost:xxxx/api/student

13. JavaScript to Call the API

```
function loadStudents() {

    fetch("/api/student")

        .then(response => response.json())

        .then(data => {

            let output = "<table>";

            output += `
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Course</th>
                    <th>Age</th>
                    <th>Email</th>
                </tr>`;

            data.forEach(student => {
```

```
                output += `
                    <tr>
                        <td>${student.id}</td>
                        <td>${student.name}</td>
                        <td>${student.course}</td>
                        <td>${student.age}</td>
                        <td>${student.email}</td>
                    </tr>`;

            });

            output += "</table>";

            document.getElementById("studentList").innerHTML = output;

        })

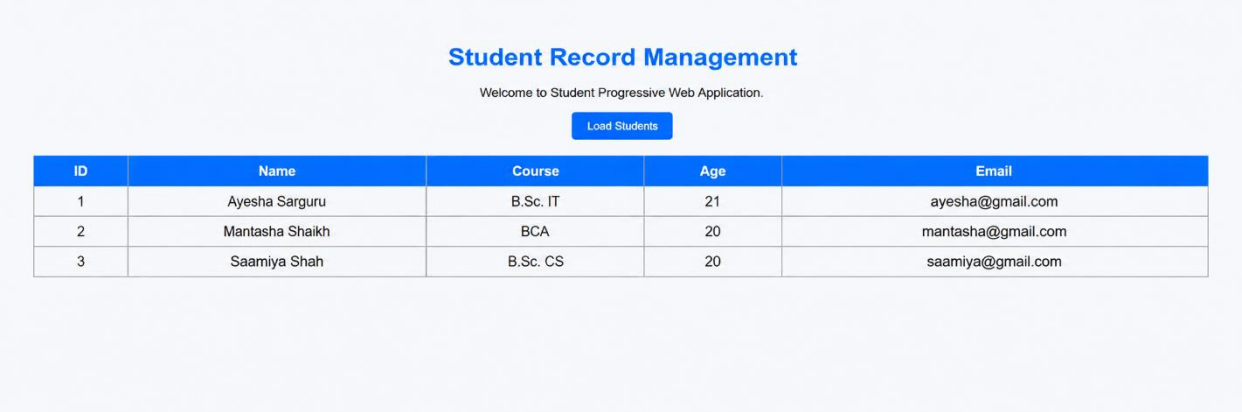
        .catch(error => {

            document.getElementById("studentList").innerHTML =
                "<p>Unable to load student data.</p>";

            console.error(error);

        });
}
```

Output:



Practical 9

ASP.NET Core Web API CRUD + Factorial API

Aim: a. Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.

b. Create an API endpoint to calculate the Factorial of a Number and test it using Postman.

Technologies

- ASP.NET Core Web API
- C#
- Entity Framework Core
- SQL Server
- Visual Studio 2026
- Postman
- .NET 10

Part A — CRUD Operations Using Web API and EF Core

We will create an Employee Management API with:

- Create Employee
- Read Employee
- Read Employee by ID
- Update Employee
- Delete Employee

The database table will be:

Employee

Id

Name

Department

Salary

Email

Step 1: Create ASP.NET Core Web API Project

Open Visual Studio 2026.

Select:

Create a new project

Choose:

ASP.NET Core Web API

Click Next.

Project name:

EmployeeWebAPI

Choose:

.NET 10.0

Authentication:

None

You can enable:

Configure for HTTPS
Click Create.

Step 2: Project Structure

After creating the project, create the following folders:
EmployeeWebAPI

```
│
├── Controllers
│   ├── EmployeeController.cs
│   └── FactorialController.cs
│
├── Data
│   └── ApplicationDbContext.cs
│
├── Models
│   └── Employee.cs
│
├── Program.cs
└── appsettings.json
```

Step 3: Install Entity Framework Core Packages

Go to:
Tools
→ NuGet Package Manager
→ Package Manager Console
Install:
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Then:
Install-Package Microsoft.EntityFrameworkCore.Tools

Step 4: Create Employee Model

Right-click the Models folder.
Select:
Add → Class
Name:
Employee.cs
Write:

```
namespace EmployeeWebAPI.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public string Name { get; set; } = "";

        public string Department { get; set; } = "";

        public decimal Salary { get; set; }

        public string Email { get; set; } = "";
    }
}
```

Explanation

Property	Description
Id	Employee ID

Property	Description
Name	Employee name
Department	Employee department
Salary	Employee salary
Email	Employee email

Step 5: Create ApplicationDbContext

Create a folder:

Data

Inside it create:

ApplicationDbContext.cs

Code:

```
using EmployeeWebAPI.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeWebAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

What is DbContext?

DbContext acts as a bridge between our C# application and SQL Server database.

C# Application

```
      ↓
Entity Framework Core
      ↓
ApplicationDbContext
      ↓
```

SQL Server

Step 6: Add Connection String

Open:

appsettings.json

Add:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeWebAPIDb;Trusted_Connection=True;TrustServerC
ertificate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
```

```
  "AllowedHosts": "*"
}
```

If you are using SQL Server Express, you can instead use something like:

```
Server=.\SQLEXPRESS;
```

Step 7: Configure Program.cs

Open:

Program.cs

Replace the code with:

```
using EmployeeWebAPI.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add Web API services
builder.Services.AddControllers();

// Add Entity Framework Core
builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlServer(
            builder.Configuration.GetConnectionString(
                "DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Step 8: Create Database Using Migration

Open:

Tools

→ NuGet Package Manager

→ Package Manager Console

Run:

Add-Migration InitialCreate

Then:

Update-Database

Entity Framework Core will create:

EmployeeWebAPIDb

with an:

Employees

table.

Step 9: Create Employee API Controller

Right-click:

Controllers

Select:

Add → Controller

You can choose an empty API controller.

Name:

EmployeeController.cs

Use the following code:

```
using EmployeeWebAPI.Data;
using EmployeeWebAPI.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace EmployeeWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public EmployeeController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/Employee
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Employee>>> GetEmployees()
        {
            return await _context.Employees.ToListAsync();
        }

        // GET: api/Employee/1
        [HttpGet("{id}")]
        public async Task<ActionResult<Employee>> GetEmployee(int id)
        {
            var employee = await  context.Employees.FindAsync(id);

            if (employee == null)
            {
                return NotFound();
            }

            return employee;
        }

        // POST: api/Employee
        [HttpPost]
        public async Task<ActionResult<Employee>> CreateEmployee(
            Employee employee)
        {
             context.Employees.Add(employee);

            await _context.SaveChangesAsync();

            return CreatedAtAction(
                nameof(GetEmployee),
                new { id = employee.Id },
                employee);
        }

        // PUT: api/Employee/1
        [HttpPut("{id}")]
        public async Task<IActionResult> UpdateEmployee(
            int id,
            Employee employee)
        {
            if (id != employee.Id)
            {
```

```
                return BadRequest();
            }

            _context.Entry(employee).State =
                EntityState.Modified;

            try
            {
                await  context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!EmployeeExists(id))
                {
                    return NotFound();
                }

                throw;
            }

            return NoContent();
        }

        // DELETE: api/Employee/1
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteEmployee(int id)
        {
            var employee =
                await  context.Employees.FindAsync(id);

            if (employee == null)
            {
                return NotFound();
            }

             context.Employees.Remove(employee);

            await _context.SaveChangesAsync();

            return NoContent();
        }

        private bool EmployeeExists(int id)
        {
            return _context.Employees
                .Any(e => e.Id == id);
        }
    }
}
```

Step 10: Understand the CRUD Endpoints

Our API now provides these endpoints:

Operation	HTTP Method	URL
Get all employees	GET	/api/Employee
Get employee	GET	/api/Employee/1
Create employee	POST	/api/Employee
Update employee	PUT	/api/Employee/1
Delete employee	DELETE	/api/Employee/1

The CRUD flow is:



```
                    ↓
                 DELETE
```

Step 11: Test GET API

Run the project.

Suppose Visual Studio gives:

https://localhost:7001

Open:

https://localhost:7001/api/Employee

Initially, because there are no records, the response will be:

[]

Step 12: Test POST Using Postman

Open Postman.

Select:

POST

URL:

https://localhost:7001/api/Employee

Go to:

Body

→ raw

→ JSON

Enter:

```
{
    "name": "Ayesha Sarguru",
    "department": "IT",
    "salary": 45000,
    "email": "ayesha@gmail.com"
}
```

Click:

Send

Expected response:

```
{
    "id": 1,
    "name": "Ayesha Sarguru",
    "department": "IT",
    "salary": 45000,
    "email": "ayesha@gmail.com"
}
```

The employee is now stored in SQL Server.

Step 13: Test GET All Employees

In Postman:

GET

URL:

https://localhost:7001/api/Employee

Click Send.

Expected response:

[

```
    {
        "id": 1,
        "name": "Ayesha Sarguru",
        "department": "IT",
        "salary": 45000,
        "email": "ayesha@gmail.com"
    }
]
```

Step 14: Test GET Employee by ID

Select:

GET

URL:

https://localhost:7001/api/Employee/1

Expected response:

```
{
    "id": 1,
    "name": "Ayesha Sarguru",
    "department": "IT",
    "salary": 45000,
    "email": "ayesha@gmail.com"
}
```

Step 15: Test PUT — Update Employee

Select:

PUT

URL:

https://localhost:7001/api/Employee/1

Body → raw → JSON:

```
{
    "id": 1,
    "name": "Ayesha Sarguru",
    "department": "HR",
    "salary": 55000,
    "email": "ayesha.sarguru@gmail.com"
}
```

Click Send.

Expected status:

204 No Content

The employee has been updated.

Step 16: Test DELETE

Select:

DELETE

URL:

https://localhost:7001/api/Employee/1

Click:

Send

Expected status:

204 No Content

The employee has been deleted.

Part B — Factorial API

Now we will create another API endpoint that calculates:

5! = 5 × 4 × 3 × 2 × 1

Therefore:

5! = 120

Step 17: Create FactorialController

Inside:

Controllers

create:

FactorialController.cs

Code:

```
using Microsoft.AspNetCore.Mvc;

namespace EmployeeWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class FactorialController : ControllerBase
    {
        [HttpGet("{number}")]
        public IActionResult CalculateFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest(
                    "Factorial is not defined for negative numbers.");
            }

            long factorial = 1;

            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }

            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}
```

Step 18: Understand the Factorial API

The route is:

```
[Route("api/[controller]")]
```

The controller name is:

FactorialController

Therefore:

api/Factorial

The method has:

```
[HttpGet("{number}")]
```

So the complete endpoint is:

GET /api/Factorial/5

Step 19: Test Factorial API in Browser

Run the application.

Open:

https://localhost:7001/api/Factorial/5

Expected response:

```
{
    "number": 5,
    "factorial": 120
}
```

For:

/api/Factorial/6

response:

```
{
    "number": 6,
    "factorial": 720
}
```

Step 20: Test Factorial API Using Postman

Open Postman.

Select:

GET

Enter:

https://localhost:7001/api/Factorial/5

Click:

Send

Expected response:

```
{
    "number": 5,
    "factorial": 120
}
```

Test another number

Request:

GET https://localhost:7001/api/Factorial/10

Response:

```
{
    "number": 10,
    "factorial": 3628800
}
```

Step 21: Test Negative Number

Try:

GET https://localhost:7001/api/Factorial/-5

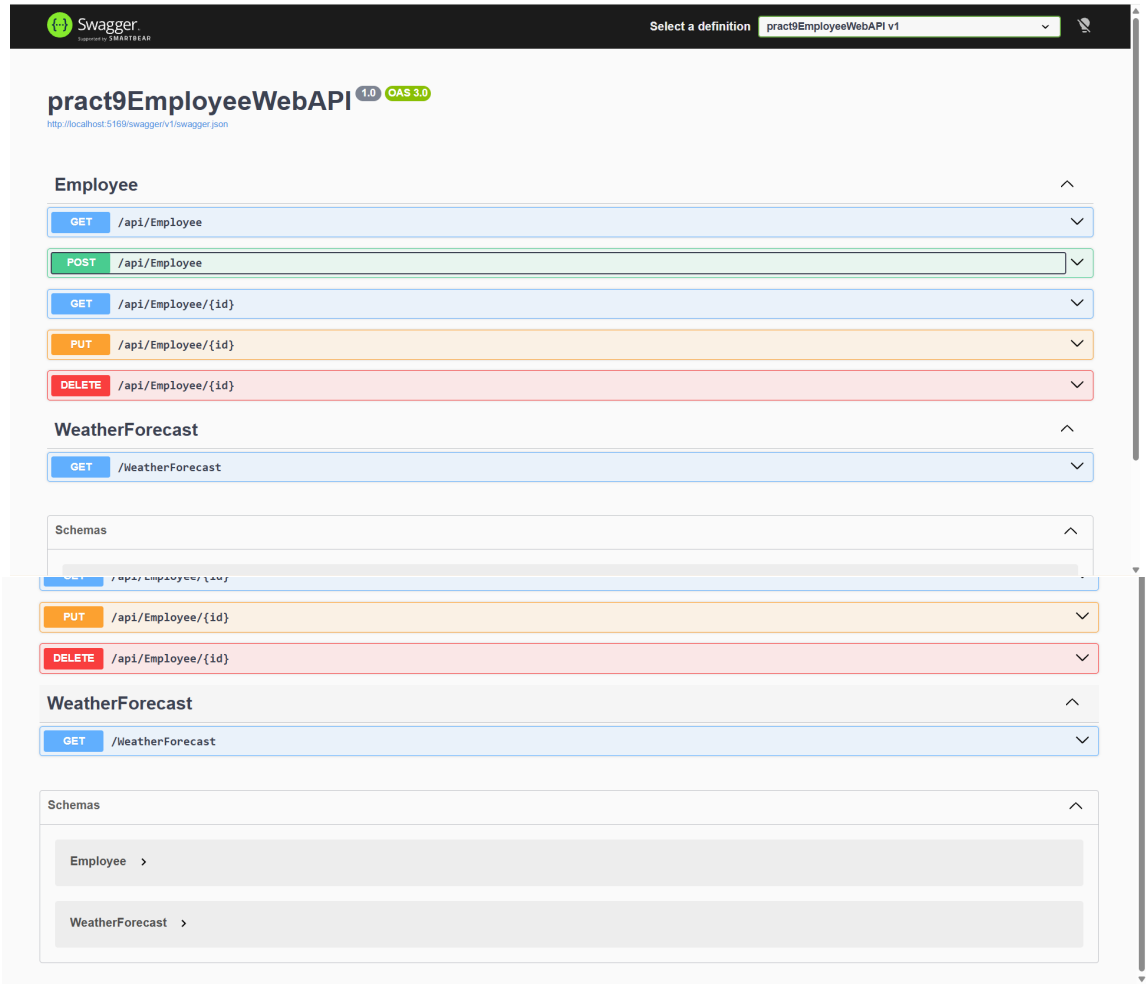
The API returns:

400 Bad Request

with:

Factorial is not defined for negative numbers.

Output:


Swagger
Select a definition
pract9EmployeeWebAPI v1
pract9EmployeeWebAPI 1.0 OAS 3.0
http://localhost:5169/swagger/v1/swagger.json
Employee
GET /api/Employee
POST /api/Employee
GET /api/Employee/{id}
PUT /api/Employee/{id}
DELETE /api/Employee/{id}
WeatherForecast
GET /WeatherForecast
Schemas
PUT /api/Employee/{id}
DELETE /api/Employee/{id}
WeatherForecast
GET /WeatherForecast
Schemas
Employee
WeatherForecast

Practical 10

Yes. Below is a complete practical-ready implementation using ASP.NET Core MVC / .NET 10, suitable for Visual Studio 2026.

Practical: PWA Manifest, Push Notifications and E-Commerce PWA

Aim:

Part A

1. Create a PWA with a Web App Manifest containing:
    - App Name
    - Icons
    - Start URL
    - Display Mode
2. Implement Push Notifications in the PWA.

Part B

Develop an E-Commerce PWA with:

- Product listing
- Add to cart
- Remove from cart
- Quantity management
- Offline cart functionality
- Service Worker caching

A real Web Push implementation requires a service worker, a browser push subscription, and an application server that sends the notification through a push service. VAPID keys are used by modern Web Push implementations. (MDN Web Docs)

1. Create the Project

Open Visual Studio 2026.

Select:

Create a new project → ASP.NET Core Web App (Model-View-Controller)

Project name:

ECommercePWA

Framework:

.NET 10.0

Authentication:

None

Click Create.

2. Project Structure

Create the following structure:

ECommercePWA

```
│
├── Controllers
│   └── HomeController.cs
│
├── Models
│   └── Product.cs
│
├── Views
│   ├── Home
```

```
│   │   └── Index.cshtml
│   └── Shared
│       └── _Layout.cshtml
│
├── wwwroot
│   ├── css
│   │   └── site.css
│   │
│   ├── js
│   │   └── site.js
│   │
│   ├── images
│   │   ├── icon-192.png
│   │   └── icon-512.png
│   │
│   ├── manifest.json
│   ├── service-worker.js
│   └── offline.html
│
├── Program.cs
└── appsettings.json
```

Important: Make sure wwwroot exists. Your previous PWA work involved static-asset problems, so verify that these files are actually inside wwwroot.

Part A — PWA Manifest

3. Create Web App Manifest

Create:

wwwroot/manifest.json

Code:

```
{
  "name": "ShopEasy E-Commerce",
  "short name": "ShopEasy",
  "start_url": "/",
  "display": "standalone",
  "background color": "#ffffff",
  "theme color": "#0d6efd",
  "description": "Offline enabled E-Commerce Progressive Web Application",

  "icons": [
    {
      "src": "/images/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/images/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Manifest Explanation

Property	Purpose
name	Complete application name
short_name	Short name displayed when space is limited
start_url	Page opened when the PWA starts
display	Controls application display mode
background_color	Background color during startup
theme_color	Browser/application theme color

Property	Purpose
icons	PWA icons

The manifest is the file that describes how a PWA should appear and integrate with the user's operating system. (MDN Web Docs)

5. Add PWA Icons

Put two PNG files inside:

wwwroot/images/

Names:

icon-192.png

icon-512.png

Recommended sizes:

icon-192.png → 192 × 192

icon-512.png → 512 × 512

You can use any suitable shopping/cart logo for the practical.

6. Add Manifest to Layout

Open:

Views/Shared/_Layout.cshtml

Inside <head>, add:

```
<link rel="manifest" href="~/manifest.json" />

<meta name="theme-color" content="#0d6efd" />

<meta name="mobile-web-app-capable" content="yes" />
```

At the bottom of <body>:

```
<script src="~/js/site.js"></script>
```

7. Create Product Model

Create:

Models/Product.cs

```
namespace ECommercePWA.Models
{
    public class Product
    {
        public int Id { get; set; }

        public string Name { get; set; } = "";

        public decimal Price { get; set; }

        public string Image { get; set; } = "";

        public string Description { get; set; } = "";
    }
}
```

8. Create HomeController

Create:

Controllers/HomeController.cs

```
using ECommercePWA.Models;
using Microsoft.AspNetCore.Mvc;
```

```
namespace ECommercePWA.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            List<Product> products = new List<Product>
            {
                new Product
                {
                    Id = 1,
                    Name = "Laptop",
                    Price = 55000,
                    Image = "laptop.jpg",
                    Description = "High performance laptop"
                },

                new Product
                {
                    Id = 2,
                    Name = "Smartphone",
                    Price = 25000,
                    Image = "phone.jpg",
                    Description = "Latest smartphone"
                },

                new Product
                {
                    Id = 3,
                    Name = "Headphones",
                    Price = 2500,
                    Image = "headphones.jpg",
                    Description = "Wireless headphones"
                },

                new Product
                {
                    Id = 4,
                    Name = "Smart Watch",
                    Price = 4500,
                    Image = "watch.jpg",
                    Description = "Fitness smart watch"
                }
            };

            return View(products);
        }
    }
}
```

9. Create Product Listing Page

Open:

Views/Home/Index.cshtml

Replace its contents with:

```
@model List<ECommercePWA.Models.Product>

@{
    ViewData["Title"] = "ShopEasy";
}

<div class="container">

    <div class="shop-header">

        <div>
            <h1>🛒 ShopEasy</h1>
            <p>Online Shopping PWA</p>
        </div>

        <div>
            <button
                class="btn btn-primary"
                onclick="showCart()">
```

```
            🛒 Cart
            <span id="cartCount" class="badge bg-danger">
                0
            </span>

        </button>

        <button
            class="btn btn-success"
            onclick="subscribeToPush()">

            🔔 Enable Notifications

        </button>
    </div>

</div>

<hr />

<h2>Products</h2>

<div class="product-grid">

    @foreach (var product in Model)
    {
        <div class="product-card">

            <img
                src="~/images/@product.Image"
                alt="@product.Name"
                onerror="this.style.display='none';" />

            <h3>@product.Name</h3>

            <p>@product.Description</p>

            <h4>
                ₹@product.Price
            </h4>

            <button
                class="btn btn-primary"
                onclick="addToCart(
                    @product.Id,
                    '@product.Name',
                    @product.Price)">

                Add to Cart

            </button>

        </div>
    }

</div>

<div id="cartSection" class="cart-section">

    <h2>🛒 Shopping Cart</h2>

    <div id="cartItems"></div>

    <h3>
        Total:
        ₹<span id="cartTotal">0</span>
    </h3>

    <button
        class="btn btn-danger"
        onclick="clearCart()">

        Clear Cart

    </button>

</div>
```

```
</div>
```

10. Add Product Images

Put product images inside:

wwwroot/images/

For example:

laptop.jpg

phone.jpg

headphones.jpg

watch.jpg

Your folder will look like:

images

```
│
├── icon-192.png
├── icon-512.png
├── laptop.jpg
├── phone.jpg
├── headphones.jpg
└── watch.jpg
```

If you don't have product images, the onerror attribute prevents the broken-image icon from affecting the product card.

11. CSS for E-Commerce Website

Open:

wwwroot/css/site.css

Add:

```
body {
    font-family: Arial, sans-serif;
    background: #f5f6fa;
}

.shop-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 20px 0;
}

.product-grid {
    display: grid;
    grid-template-columns:
        repeat(auto-fit, minmax(220px, 1fr));

    gap: 20px;
    margin-top: 20px;
}

.product-card {
    background: white;
    padding: 20px;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

.product-card img {
    width: 150px;
    height: 150px;
    object-fit: contain;
}

.product-card h3 {
    margin-top: 15px;
```

```
}

.cart-section {
    margin-top: 40px;
    padding: 25px;
    background: white;
    border-radius: 10px;
}

.cart-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 12px;
    border-bottom: 1px solid #ddd;
}
```

12. Implement Offline Cart

The important part of this practical is that the cart should continue to work without internet.

We will use:

localStorage

The browser stores the cart locally.

Create/open:

wwwroot/js/site.js

Use:

```
let cart =
    JSON.parse(localStorage.getItem("shoppingCart")) || [];

document.addEventListener("DOMContentLoaded", function () {

    updateCart();

    registerServiceWorker();

});


// Add product to cart
function addToCart(id, name, price) {

    let existingProduct =
        cart.find(product => product.id === id);

    if (existingProduct) {

        existingProduct.quantity++;

    }
    else {

        cart.push({
            id: id,
            name: name,
            price: price,
            quantity: 1
        });

    }

    saveCart();

    updateCart();

    alert(name + " added to cart!");
}
```

// Save cart in Local Storage

```
function saveCart() {
```

```
    localStorage.setItem(
        "shoppingCart",
        JSON.stringify(cart)
    );

}


// Display cart
function updateCart() {

    const cartItems =
        document.getElementById("cartItems");

    const cartCount =
        document.getElementById("cartCount");

    const cartTotal =
        document.getElementById("cartTotal");

    if (!cartItems) return;

    cartItems.innerHTML = "";

    let total = 0;
    let count = 0;

    cart.forEach(product => {

        const productTotal =
            product.price * product.quantity;

        total += productTotal;

        count += product.quantity;

        cartItems.innerHTML += `

            <div class="cart-item">

                <div>
                    <strong>${product.name}</strong>
                    <br>

                    ₹${product.price}
                    × ${product.quantity}

                </div>

                <div>

                    <button
                        class="btn btn-sm btn-secondary"
                        onclick="decreaseQuantity(${product.id})">

                        -

                    </button>

                    <button
                        class="btn btn-sm btn-secondary"
                        onclick="increaseQuantity(${product.id})">

                        +

                    </button>

                    <button
                        class="btn btn-sm btn-danger"
                        onclick="removeFromCart(${product.id})">

                        Remove

                    </button>

                </div>

            </div>
```

```
        `;

    });

    cartCount.textContent = count;

    cartTotal.textContent = total;

}


// Increase quantity
function increaseQuantity(id) {

    const product =
        cart.find(product => product.id === id);

    if (product) {

        product.quantity++;

    }

    saveCart();

    updateCart();

}


// Decrease quantity
function decreaseQuantity(id) {

    const product =
        cart.find(product => product.id === id);

    if (!product) return;

    product.quantity--;

    if (product.quantity <= 0) {

        cart =
            cart.filter(product => product.id !== id);

    }

    saveCart();

    updateCart();

}


// Remove product
function removeFromCart(id) {

    cart =
        cart.filter(product => product.id !== id);

    saveCart();

    updateCart();

}


// Clear cart
function clearCart() {

    if (confirm("Are you sure you want to clear the cart?")) {

        cart = [];

        saveCart();

        updateCart();
```

```
    }

}


// Show cart
function showCart() {

    const section =
        document.getElementById("cartSection");

    section.scrollIntoView({
        behavior: "smooth"
    });

}


// Register Service Worker
function registerServiceWorker() {

    if ("serviceWorker" in navigator) {

        navigator.serviceWorker
            .register("/service-worker.js")
            .then(function (registration) {

                console.log(
                    "Service Worker registered:",
                    registration.scope
                );

            })
            .catch(function (error) {

                console.error(
                    "Service Worker registration failed:",
                    error
                );

            });

    }

}
```

13. How Offline Cart Works

The important concept is:

```
                E-Commerce PWA
                      |
          ┌───────────┴──────────┐
          ↓                      ↓
    Service Worker          Local Storage
          ↓                      ↓
    Cache Website            Cart Data
          ↓                      ↓
       Offline                Offline
      Application              Cart
```

For example:

Online

↓

User adds Laptop

↓

Cart saved in localStorage

↓

Internet disconnected

↓

PWA still opens from cache

↓

Cart is read from localStorage

↓

User can add/remove products

14. Create Service Worker

Create:

wwwroot/service-worker.js

Code:

```
const CACHE_NAME = "shopeasy-pwa-v1";

const FILES_TO_CACHE = [
    "/",
    "/manifest.json",
    "/css/site.css",
    "/js/site.js",
    "/offline.html"
];


self.addEventListener("install", event => {

    event.waitUntil(

        caches.open(CACHE_NAME)
            .then(cache => {

                return cache.addAll(FILES_TO_CACHE);

            })

    );

    self.skipWaiting();

});


self.addEventListener("activate", event => {

    event.waitUntil(

        caches.keys()
            .then(cacheNames => {

                return Promise.all(

                    cacheNames
                        .filter(name =>
                            name !== CACHE_NAME
                        )

                        .map(name =>
                            caches.delete(name)
                        )

                );

            })

    );

    self.clients.claim();

});


self.addEventListener("fetch", event => {

    event.respondWith(
```

```
        fetch(event.request)
            .then(response => {

                return response;

            })

            .catch(() => {

                return caches.match(event.request)

                    .then(response => {

                        if (response) {

                            return response;

                        }

                        return caches.match(
                            "/offline.html"
                        );

                    });

            })

    );

});
```

15. Create Offline Page

Create:

wwwroot/offline.html

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Offline - ShopEasy</title>

    <style>

        body {
            font-family: Arial;
            text-align: center;
            padding-top: 100px;
            background: #f5f6fa;
        }

        h1 {
            color: #dc3545;
        }

        p {
            font-size: 18px;
        }

    </style>

</head>

<body>

    <h1>📡 You are Offline</h1>

    <p>
        Don't worry! Your shopping cart is still available.
```

```
    </p>

    <p>
        Please reconnect to the internet to continue shopping.
    </p>

</body>

</html>
```

Part A — Push Notifications

Now we implement actual Web Push.

A service worker must be active before the browser can create a PushSubscription. The subscription contains an endpoint and encryption information that the application server uses to send notifications. (MDN Web Docs)

16. Install WebPush Package

Open:

Tools

→ NuGet Package Manager

→ Package Manager Console

Run:

Install-Package WebPush

The C# WebPush library provides WebPushClient, PushSubscription, VapidDetails, and VAPID key generation. (GitHub)

17. Generate VAPID Keys

Create a temporary C# console program or use the following code once:

```
using WebPush;

var vapidKeys = VapidHelper.GenerateVapidKeys();

Console.WriteLine("Public Key:");
Console.WriteLine(vapidKeys.PublicKey);

Console.WriteLine("Private Key:");
Console.WriteLine(vapidKeys.PrivateKey);
```

You will receive:

Public Key:

BXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Private Key:

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Do not expose the private key to browser JavaScript.

---

18. Add VAPID Configuration

In:

appsettings.json

add:

```
{
  "VapidKeys": {
    "Subject": "mailto:your-email@example.com",
    "PublicKey": "YOUR_PUBLIC_VAPID_KEY",
    "PrivateKey": "YOUR_PRIVATE_VAPID_KEY"
```

```
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

Replace:
YOUR_PUBLIC_VAPID_KEY
YOUR_PRIVATE_VAPID_KEY
with the keys you generated.

19. Create Push Subscription Model

Create:
Models/PushSubscriptionModel.cs

```
namespace ECommercePWA.Models
{
    public class PushSubscriptionModel
    {
        public string Endpoint { get; set; } = "";

        public PushKeys Keys { get; set; } = new();
    }

    public class PushKeys
    {
        public string P256dh { get; set; } = "";

        public string Auth { get; set; } = "";
    }
}
```

20. Create Push Controller

Create:
Controllers/PushController.cs

```
using Microsoft.AspNetCore.Mvc;
using WebPush;
using ECommercePWA.Models;

namespace ECommercePWA.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PushController : ControllerBase
    {
        private readonly IConfiguration _configuration;

        public PushController(IConfiguration configuration)
        {
            _configuration = configuration;
        }

        [HttpGet("public-key")]
        public IActionResult GetPublicKey()
        {
            var publicKey =
                _configuration["VapidKeys:PublicKey"];

            return Ok(new
            {
                publicKey = publicKey
            });
```

```
        }

        [HttpPost("send")]
        public async Task<IActionResult> SendNotification(
            PushSubscriptionModel model)
        {
            var subscription =
                new PushSubscription(
                    model.Endpoint,
                    model.Keys.P256dh,
                    model.Keys.Auth
                );

            var vapidDetails =
                new VapidDetails(
                    _configuration["VapidKeys:Subject"],
                    _configuration["VapidKeys:PublicKey"],
                    _configuration["VapidKeys:PrivateKey"]
                );

            var payload = """
            {
                "title": "ShopEasy",
                "body": "New offers are available!",
                "url": "/"
            }
            """;

            var webPushClient =
                new WebPushClient();

            try
            {
                await webPushClient.SendNotificationAsync(
                    subscription,
                    payload,
                    vapidDetails
                );

                return Ok(new
                {
                    message = "Notification sent successfully."
                });
            }
            catch (WebPushException ex)
            {
                return StatusCode(
                    (int)ex.StatusCode,
                    new
                    {
                        message =
                            "Unable to send notification.",
                        error = ex.Message
                    });
            }
        }
    }
}
```

21. Update Service Worker for Push

Now replace the service worker with:

```
const CACHE_NAME = "shopeasy-pwa-v1";

const FILES_TO_CACHE = [
    "/",
    "/manifest.json",
    "/css/site.css",
    "/js/site.js",
    "/offline.html"
];


self.addEventListener("install", event => {

    event.waitUntil(
```

```
        caches.open(CACHE_NAME)
            .then(cache =>
                cache.addAll(FILES_TO_CACHE)
            )

    );

    self.skipWaiting();

});


self.addEventListener("activate", event => {

    event.waitUntil(

        caches.keys()
            .then(cacheNames => {

                return Promise.all(

                    cacheNames
                        .filter(name =>
                            name !== CACHE NAME
                        )

                        .map(name =>
                            caches.delete(name)
                        )

                );

            })

    );

    self.clients.claim();

});


self.addEventListener("fetch", event => {

    event.respondWith(

        fetch(event.request)
            .catch(() => {

                return caches.match(event.request)
                    .then(response => {

                        if (response) {
                            return response;
                        }

                        return caches.match(
                            "/offline.html"
                        );

                    });

            })

    );

});


/* PUSH NOTIFICATION */

self.addEventListener("push", event => {

    let data = {
        title: "ShopEasy",
        body: "You have a new notification!",
        url: "/"
    };

    if (event.data) {
```

```
        try {

            data = event.data.json();

        }
        catch {

            data.body = event.data.text();

        }

    }

    const options = {

        body: data.body,

        icon: "/images/icon-192.png",

        badge: "/images/icon-192.png",

        data: {
            url: data.url
        }

    };

    event.waitUntil(

        self.registration.showNotification(
            data.title,
            options
        )

    );

});


/* NOTIFICATION CLICK */

self.addEventListener(
    "notificationclick",
    event => {

        event.notification.close();

        const url =
            event.notification.data.url || "/";

        event.waitUntil(

            clients.matchAll({
                type: "window",
                includeUncontrolled: true
            })

            .then(windowClients => {

                for (const client
                    of windowClients) {

                    if (
                        client.url.includes(url)
                        &&
                        "focus" in client
                    ) {

                        return client.focus();

                    }

                }

                if (clients.openWindow) {

                    return clients.openWindow(url);
```

```
                }

            })

        );

    }
);
The push event is handled by the service worker, which can then display a notification using
showNotification(). (MDN Web Docs)
```

22. Add Push Subscription JavaScript

Add this to:

wwwroot/js/site.js

```
async function subscribeToPush() {

    if (!("serviceWorker" in navigator)) {

        alert("Service Worker is not supported.");

        return;

    }

    if (!("PushManager" in window)) {

        alert("Push notifications are not supported.");

        return;

    }

    const permission =
        await Notification.requestPermission();

    if (permission !== "granted") {

        alert("Notification permission was denied.");

        return;

    }

    try {

        const registration =
            await navigator.serviceWorker.ready;

        const response =
            await fetch("/api/push/public-key");

        const data =
            await response.json();

        const applicationServerKey =
            urlBase64ToUint8Array(data.publicKey);

        let subscription =
            await registration.pushManager
                .getSubscription();

        if (!subscription) {

            subscription =
                await registration.pushManager.subscribe({

                    userVisibleOnly: true,

                    applicationServerKey:
                        applicationServerKey

                });

        }

        await fetch("/api/push/send", {
```

```
            method: "POST",

            headers: {
                "Content-Type": "application/json"
            },

            body: JSON.stringify(subscription)

        });

        alert(
            "Push notification subscription successful!"
        );

    }
    catch (error) {

        console.error(error);

        alert(
            "Unable to subscribe to push notifications."
        );

    }
}


function urlBase64ToUint8Array(base64String) {

    const padding =
        "=".repeat(
            (4 - base64String.length % 4) % 4
        );

    const base64 =
        (
            base64String +
            padding
        )
        .replace(/-/g, "+")
        .replace(/_/g, "/");

    const rawData =
        window.atob(base64);

    return Uint8Array.from(
        [...rawData]
            .map(char => char.charCodeAt(0))
    );

}
```

23. Program.cs

Make sure your Program.cs contains:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();
```

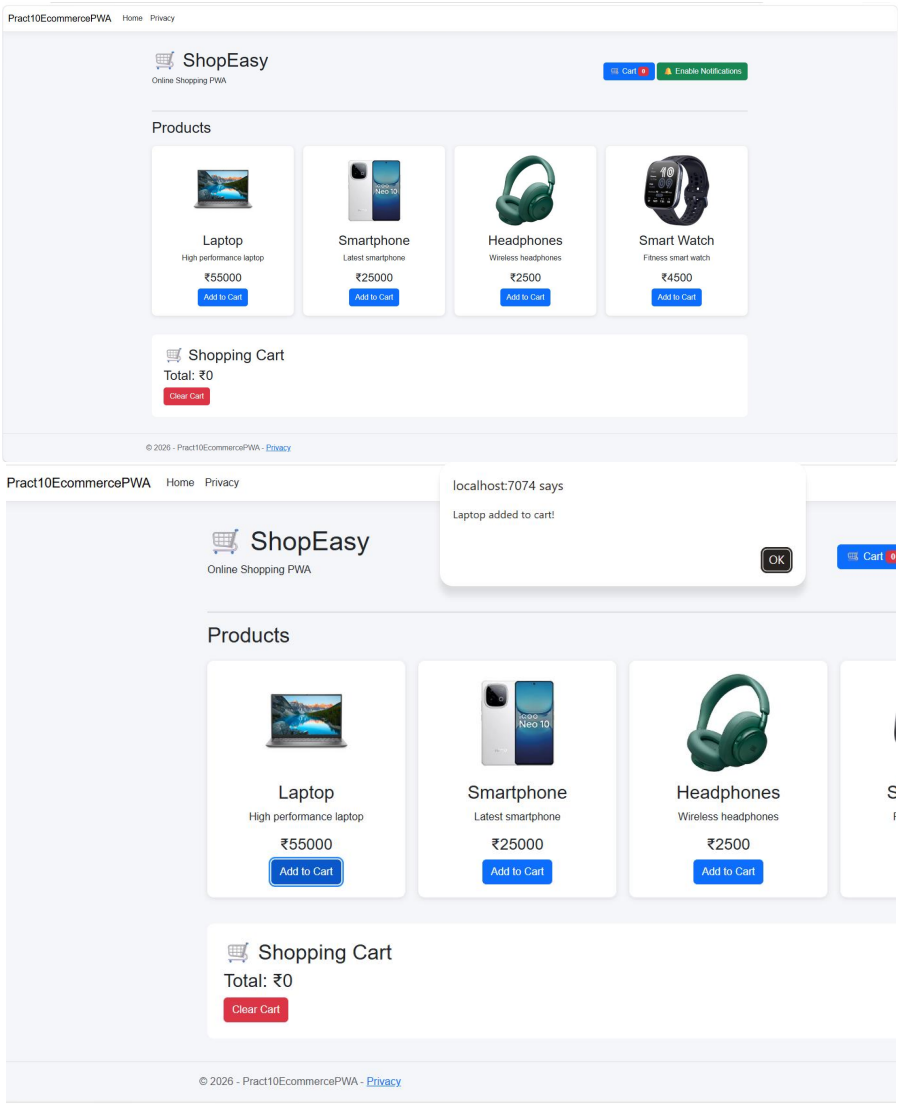
```
app.UseAuthorization();

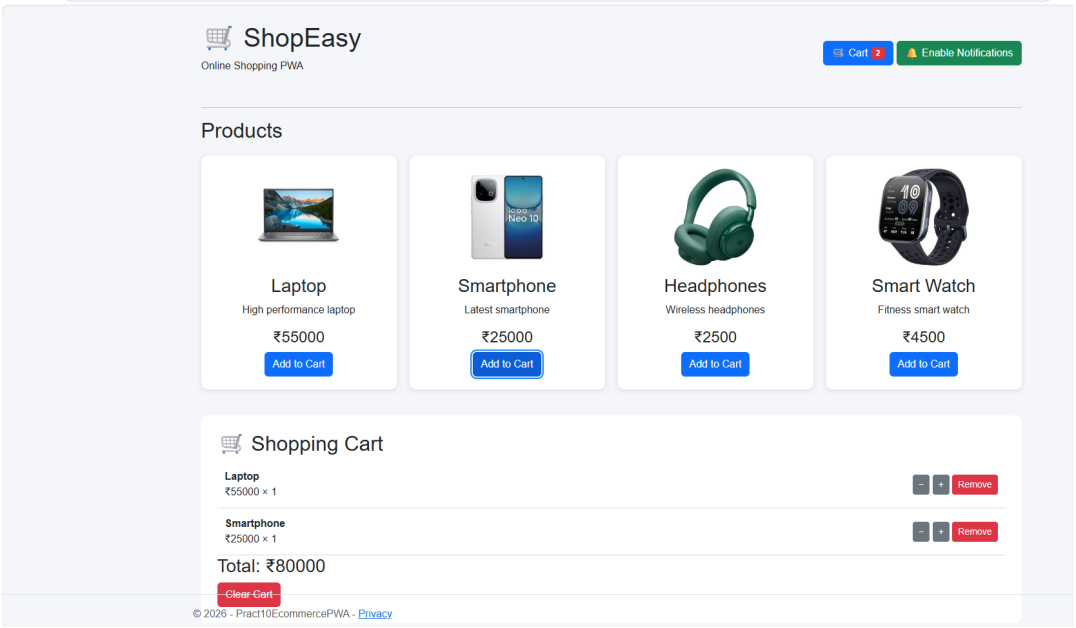
app.MapControllers();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Output:




ShopEasy
Online Shopping PWA
Cart 2
Enable Notifications
Products
Laptop
High performance laptop
₹55000
Add to Cart
Smartphone
Latest smartphone
₹25000
Add to Cart
Headphones
Wireless headphones
₹2500
Add to Cart
Smart Watch
Fitness smart watch
₹4500
Add to Cart
Shopping Cart
Laptop
₹55000 × 1
Remove
Smartphone
₹25000 × 1
Remove
Total: ₹80000
Clear Cart
© 2026 - Pract10EcommercePWA - Privacy